ADVANCED

DEPLOYMENT STRATEGIES



learnk8s

Table of contents

1. Is the app down for everyone or just me? 5

Application health checks 9

Readiness probe 9

Liveness probe 10

Implementing readiness and liveness probes 11

Liveness and readiness probes in action 15

Rolling updates 20

Canary deployments 24

Using selectors and labels 27

Using labels and selectors with Canary Deployments 29

Blue-green deployments 35

Rolling back a Deployment 37

2. Before you start 55

Familiarise with the apps 57

3. Rolling update strategy 61

Adding a Liveness probe 65

Adding a readiness probe 68
Rolling updates 75
Monitoring the rolling updates 76
Tidying up 79

4. Services and selectors 81

Deploying the applications 82
Deploying version 1.0.0 82
Deploying version 2.0.0 83
Distributing traffic to one app at the time 86
Distributing traffic to both apps 88
Labels challenge 90
Tidying up 92

5. Canary deployments 93

Deploying the production application 95
Deploying the canary 98
Monitoring the live traffic 100
Exposing the canary deployment 101

Balancing the split 103

Multiple canaries challenge 104

Tidying up 108

6. Blue-green deployments 109

Deploying the production application 111

Monitoring 114

Rainbow deployment challenge 118

Tidying up 119

7. Rolling back a deployment 121

Creating a deployment 123

Chapter 1

Is the app down for everyone or just me?

When you deploy an application, how do you know if the application was deployed correctly and can respond to live traffic?

You might be tempted to visit the application after each deployment to make sure that it works the way you expect.

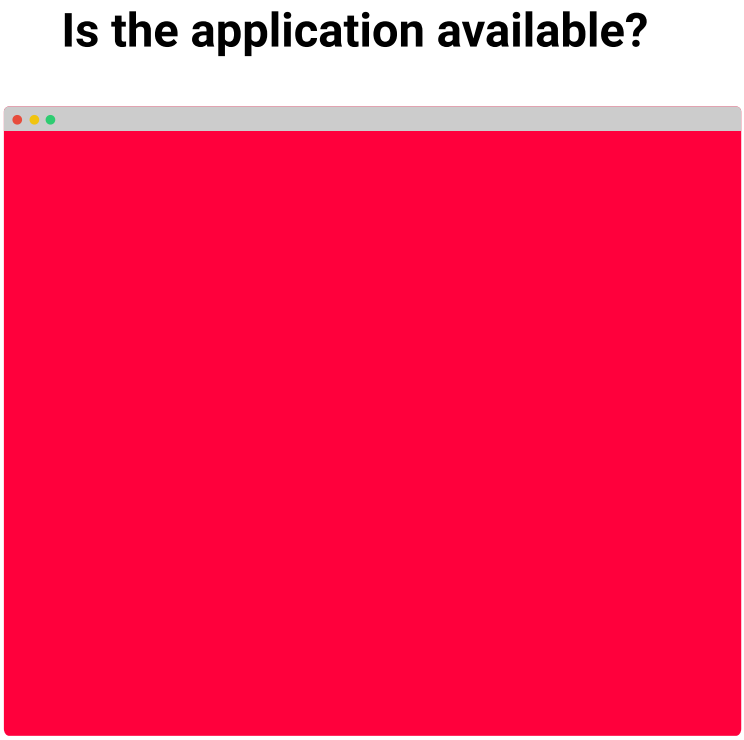


Fig. Is the app deployed successfully?

If you have a single application, checking the status after each deployment is annoying but doable.

When your application becomes popular, you can scale the app to three instances and still check that the deployment was successful by visiting each app.

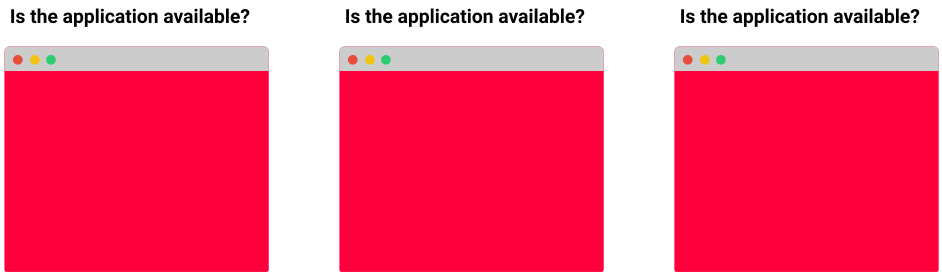


Fig. Are the three apps deployed successfully?

It's harder but still manageable.

If you have hundreds of apps and dozens of instances each, it becomes a full-time job to check the status of each one of them individually.

It doesn't scale.



Fig. Checking hundreds of apps for uptime doesn't scale

It would be much better if instead of you policing the deployment, you could have the app reporting the status back to you.

Instead of probing for a successful response, you could have an automated process scraping the health of your Pods.

If the app is unhealthy perhaps you could try to restart it —

maybe to fix a transient bug — or you could at least remove it from the Service if it's returning errors as a response.

You don't want unhealthy replicas replying with errors to production traffic.

Application health checks

Kubernetes offers two mechanisms to track the lifecycle of your containers and Pods: **liveness and readiness probes**.

Readiness probe

A readiness probe is designed to probe the containers inside a Pod and decide if they can receive traffic.

The probe attaches and detaches the container to the Service when it detects a healthy or unhealthy response from the container.

Imagine having a web application written in Spring Boot.

Depending on the resources allocated to it, it could take about 30 seconds for the application to be ready to serve traffic.

You don't want to serve traffic to that app until it's entirely bootstrapped.

The readiness probe is designed to solve such a problem.

You can add a readiness probe to your containers like this:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: readiness-pod
spec:
  containers:
  - name: sise
    image: learnk8s/demo:1.0.0
    ports:
    - containerPort: 3000
    readinessProbe:
      initialDelaySeconds: 2
      periodSeconds: 5
      httpGet:
        path: /ready
        port: 3000
```

Liveness probe

Kubernetes has a second probe called the liveness probe.

The liveness probe is designed to monitor your container and restart it when unhealthy.

When you have a memory leak in one of your containers, the application could reject incoming requests or return invalid responses.

You could have a readiness probe set up to detach the unhealthy container from the Service.

However, the container (and the Pod) will still be counted against the total replicas for your Deployment.

If you have a Deployment with three replicas with a single

container, one will be unable to receive traffic and Kubernetes won't take any further action.

Unless the liveness probe is failing.

In such a scenario, Kubernetes tries to restart the container inside the Pod in the hope that could be enough to fix the issue.

You can add a liveness probe to your containers like this:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: readiness-pod
spec:
  containers:
  - name: sise
    image: learnk8s/demo:1.0.0
    ports:
    - containerPort: 3000
    livenessProbe:
      initialDelaySeconds: 2
      periodSeconds: 5
      httpGet:
        path: /health
        port: 3000
```

Implementing readiness and liveness probes

Liveness and readiness probes are entirely independent.

You could have a readiness probe but not liveness.

Or vice versa. Or none of them. Or both.

When you have both, they are still independent.

The liveness probe doesn't wait for a readiness probe to detach the container from the Service before restarting it.

If you're not careful, you could have the container restarting while it's still attached to the Service and it's receiving live traffic.

Most of the time, you will expose an HTTP endpoint from your container for each of your probes.

It's common to have a `/health` and `/ready` API endpoints returning a 200 when the container is healthy and ready respectively.

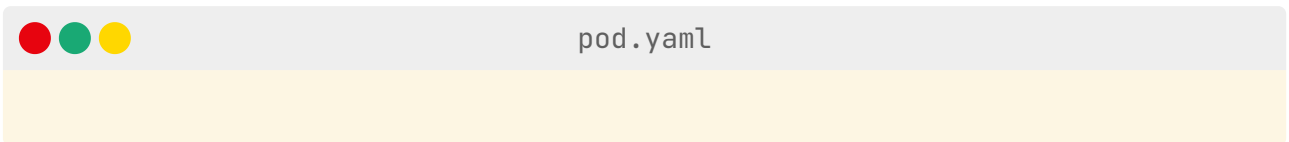
However, it doesn't always make sense to have HTTP endpoints.

If you package a MySQL database inside a container, you don't want to add a web server with two endpoints just for the readiness and liveness probes.

You don't have to.

You can tell Kubernetes to execute a command inside the container rather than requesting an HTTP endpoint.

In the case of your MySQL container, you could have a readiness probe like this:



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:5.7
    readinessProbe:
      exec:
        command: ["mysql", "-h", "127.0.0.1", "-e", "SELECT 1"]
      initialDelaySeconds: 5
      periodSeconds: 2
      timeoutSeconds: 1
```

The probe executes a command inside the container to check if the database is ready.

Liveness and readiness probes will check the status of your containers at regular intervals.

You can customise how frequently each probe should run and if it should wait for a delay before checking it the first time.

You can also tailor how many successful responses you should receive before considering the container healthy or unhealthy.

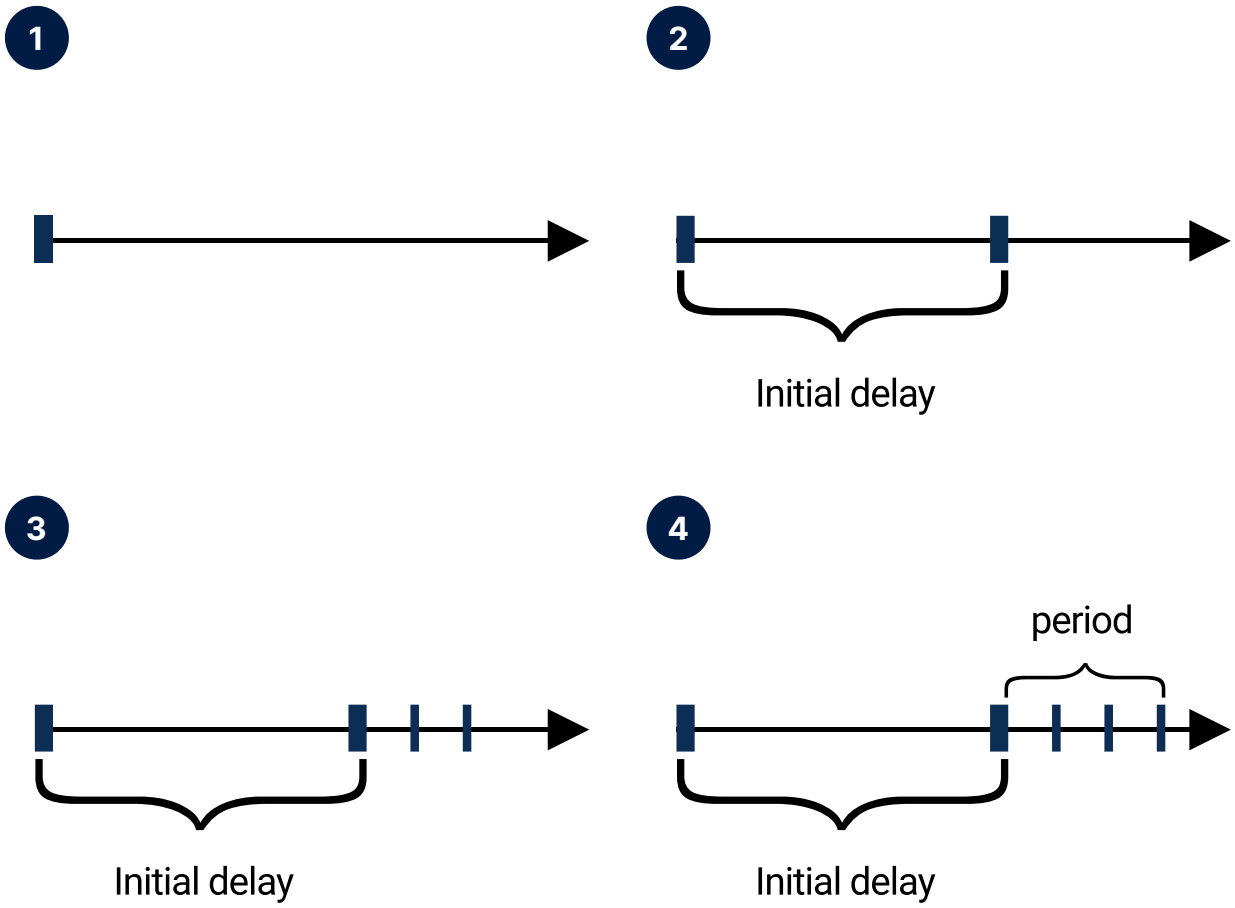


Fig. 1 Probes start monitoring your containers when your Pod starts.

Fig. 2 If you know that the application isn't ready, and it doesn't make sense to check the containers, you can set an initial delay.

Fig. 3 After the initial delay, the probe checks the container at regular intervals (defaults to 10 seconds).

Fig. 4 You can change the period to any value that is greater or equal than 1 second.

You can also customise:

- the number of seconds after which the probe times out with `timeoutSeconds`
- the minimum consecutive successes for the probe to be considered successful after having failed with `successThreshold`
- the number of successive retries before considering a probe as failed `failureThreshold`

Liveness and readiness probes in action

Let's have a look at an example of liveness and readiness for the following deployment.

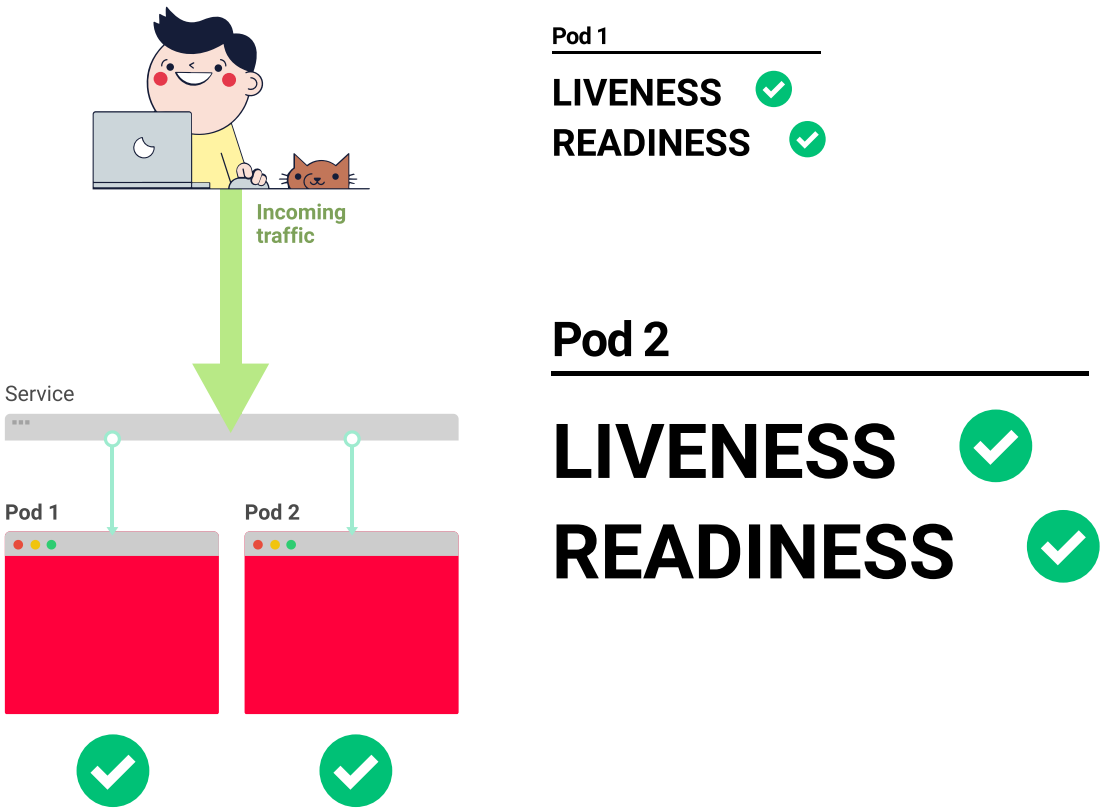


Fig. Deployment

The deployment has:

- Two replicas.
- One service.
- Each Pod has a single container with a liveness and readiness probe.

What happens when the readiness is failing?

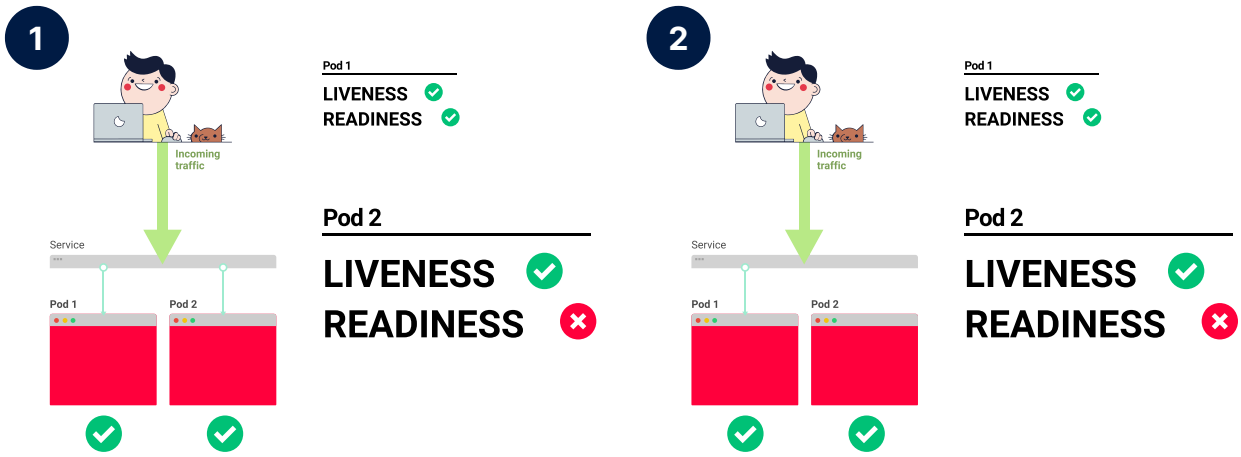


Fig. 1 If the application is unable to handle traffic, you could have it return a failing readiness probe.

Fig. 2 Kubernetes monitor the response for the readiness probe and detaches the application from the Service. No traffic reaches that container. (Please notice the missing link in the diagram)

Readiness probes are great to stop receiving traffic, which is generally useful when:

- There's an error in the app and you want to stop receiving traffic.
- You are still initialising the app and you are not ready to receive traffic yet.

The liveness is probe is useful to exit from unrecoverable states.

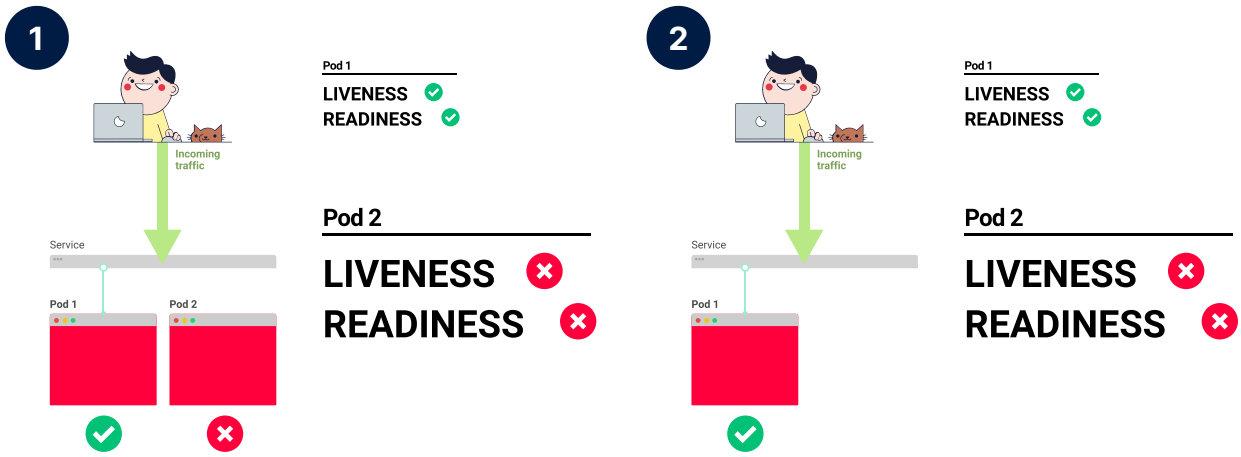


Fig. 1 Let's imagine that the app found itself stuck in an infinite loop. There's nothing you can do apart from restarting it.

Fig. 2 When the liveness probe is failing, Kubernetes restarts the container.

Liveness probes are useful for scenarios where there's nothing left but restarting the app such as:

- A deadlock.
- An infinite loop.

What happens after the container in the Pod is restarted?

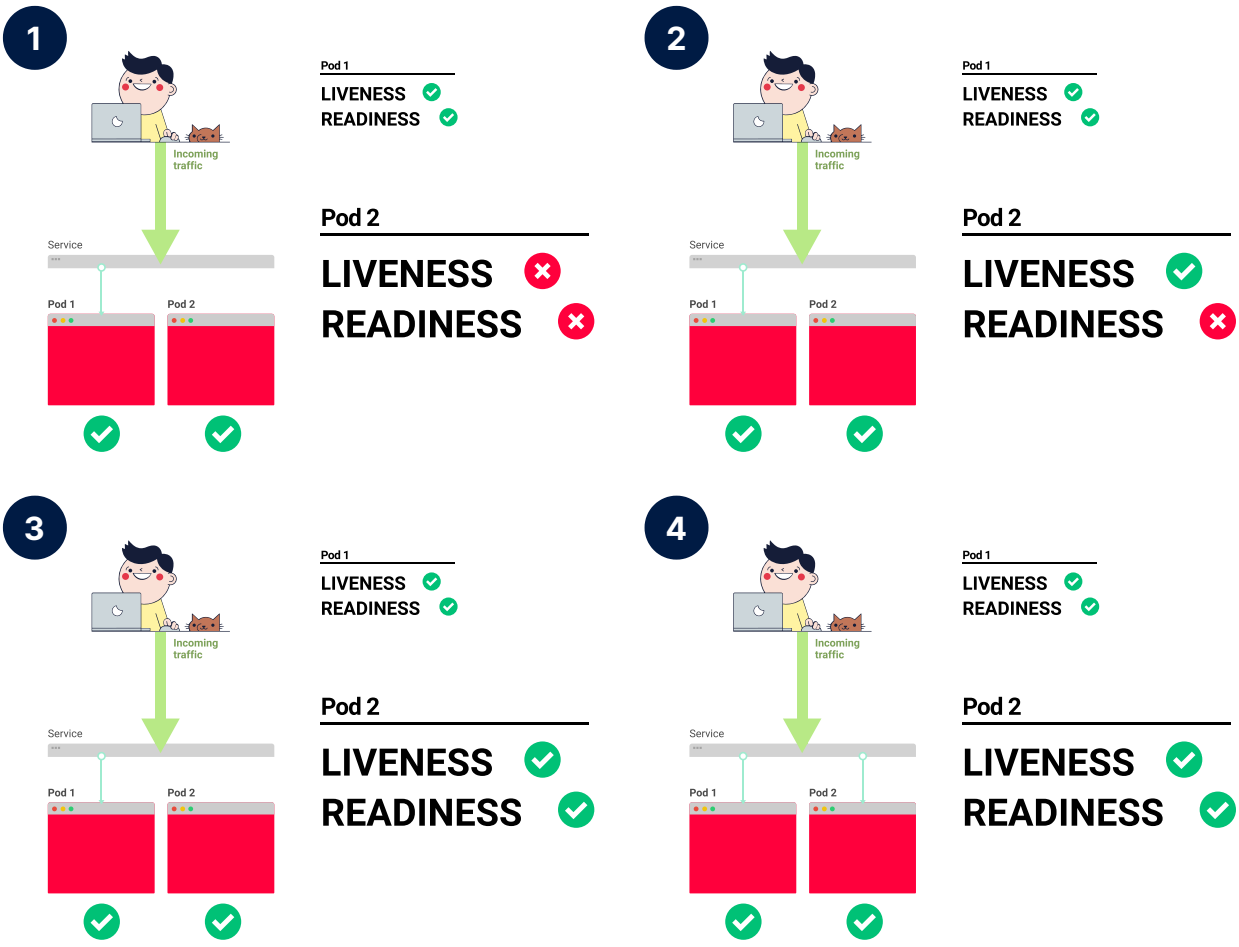


Fig. 1 The container comes back up and looks healthy. The liveness and readiness probe are healthy too.

Fig. 2 Since the liveness probe is successful, Kubernetes doesn't restart the container further.

Fig. 3 And since the readiness probe is healthy, Kubernetes reattaches the container to the load balancer.

Fig. 4 Please notice the added link between the Pod and Service in the diagram.

Readiness and liveness probes are even more critical when you decide to ship features to production.

Rolling updates

Let's imagine you have your application deployed in Kubernetes.

You want to upgrade the app to version 2.0, and you wish to do so with a zero-downtime upgrade.



Fig. Two versions of the same application

The easier option at your disposal is to add the newer version of the app alongside the existing ones and take out an old one.

If you repeat the same steps for all instances in your ReplicaSet, you will eventually migrate your production workload from version one to version two.

Replacing Pods one at a time is usually referred to as a

rolling update.
Let's have a look at an example:

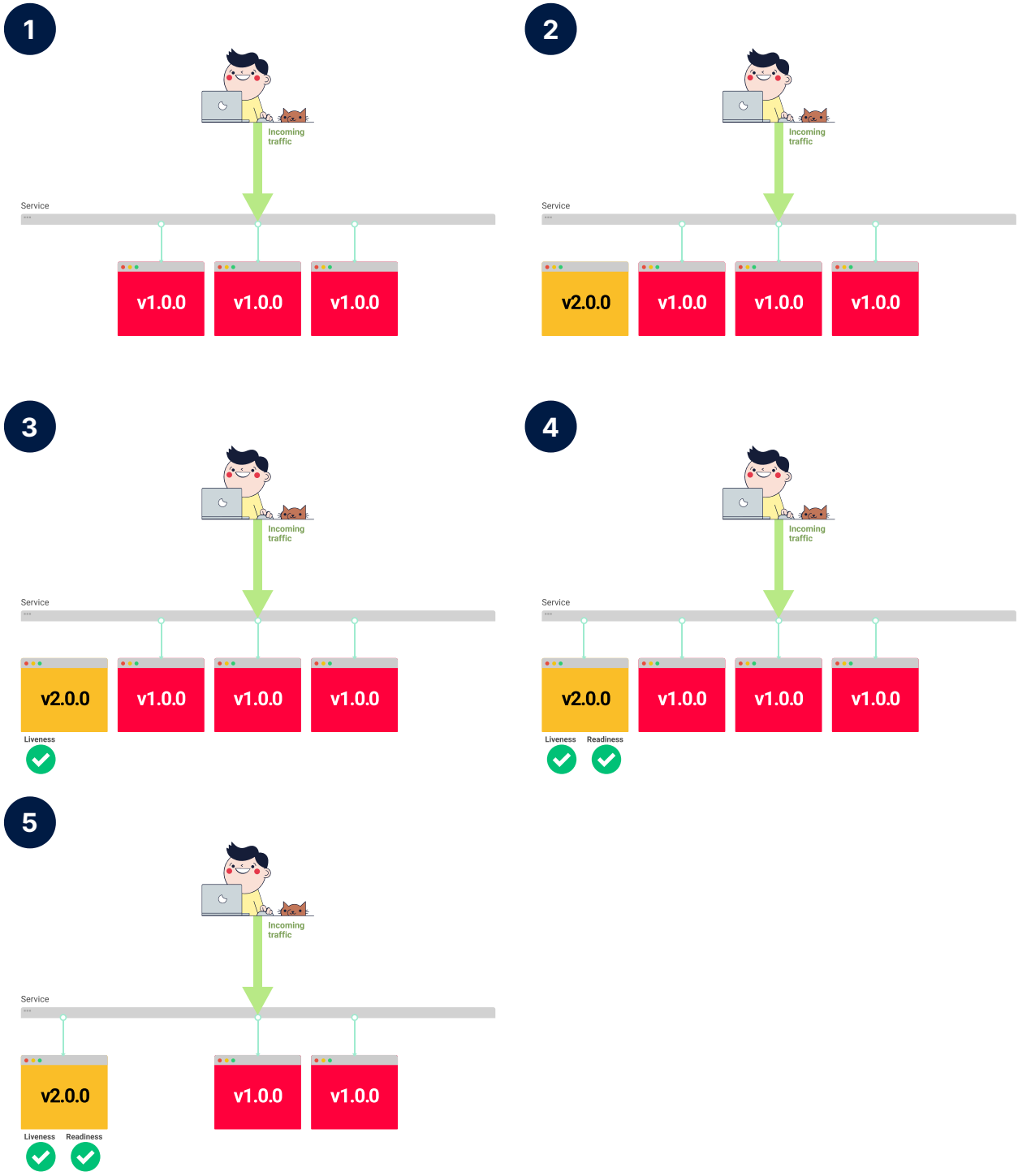


Fig. 1 You have a Service and a Deployment with three replicas on version `1.0.0`. You change the `image` in your Deployment to version `2.0.0`, here's what happens next.

Fig. 2 In a rolling update, Kubernetes create a Pod with a new version of the image.

Fig. 3 Kubernetes waits for readiness and liveness probe. When both are healthy, the Pod is running and can receive traffic.

Fig. 4 Kubernetes waits for readiness and liveness probe. When both are healthy, the Pod is running and can receive traffic.

Fig. 5 The previous Pod is removed and Kubernetes is ready to start again. The process is repeated for every Pod in the Deployment until all Pods are on version `2.0.0`.

In the rolling update pictured above, you have three replicas of the same Pod.

When you decide to change your image version in your deployment and use version two, this is what happens.

- Kubernetes will create a new Pod with the latest version
- Kubernetes will wait for the liveness probe to be healthy

- As soon as the readiness probe passes, the Pod is attached to the Service and is ready to receive traffic the Pod is ready
- Kubernetes will take down one of the Pods in the Deployment

The same steps are repeated for every other Pod in the Deployment until the rollout is completed.

Rolling updates are excellent when you wish to deliver features in production incrementally.

Newer instances gradually serve live traffic.

Beware that at any point in time, your user could receive traffic from either the current or previous version.

Rolling updates are not a suitable rollout strategy if you have breaking changes.

Canary deployments

Another option to deploy to production without disrupting live traffic is to use a Canary deployment.

Canary deployments are similar to rolling upgrades in the sense that you have your live traffic hitting the current and previous version of the app.

But whereas with a rolling update the upgrades are executed

one after the other, **with a canary deployment you have two versions of your app (current and previous) deployed at the same time.**

It's common practice to create only a few Pods with the current version and monitor the live traffic reaching them.

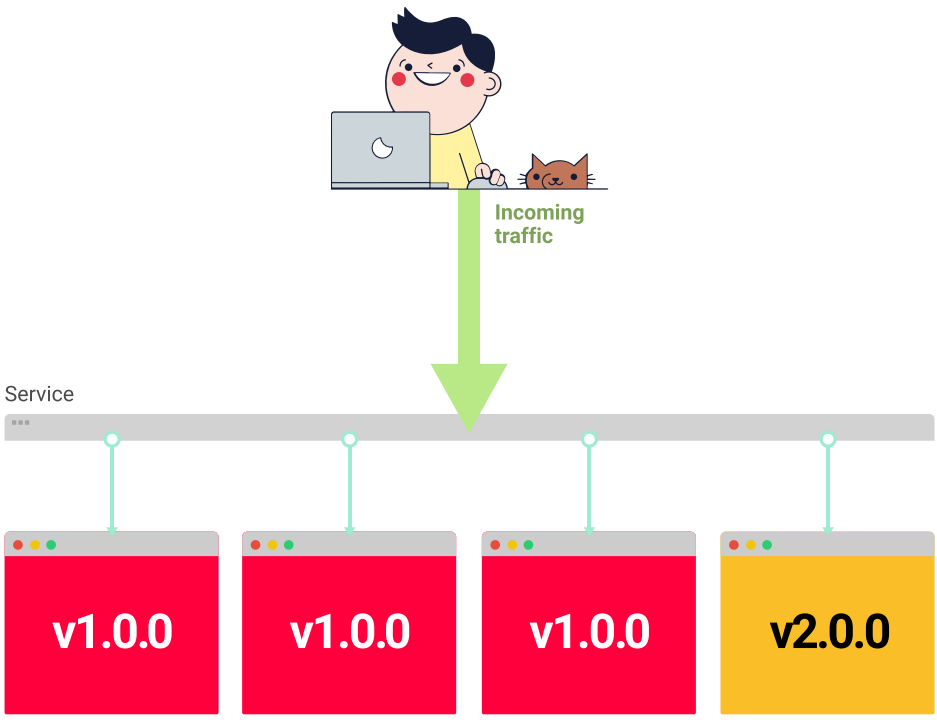


Fig. In a Canary deployment, previous and current app are available at the same time

In this example, 25% of the traffic hits the current version

and 75% the previous.

If everything goes as planned, you can increase the number of replicas in your current version and gradually serve more traffic while you decrease the replicas for the previous app.

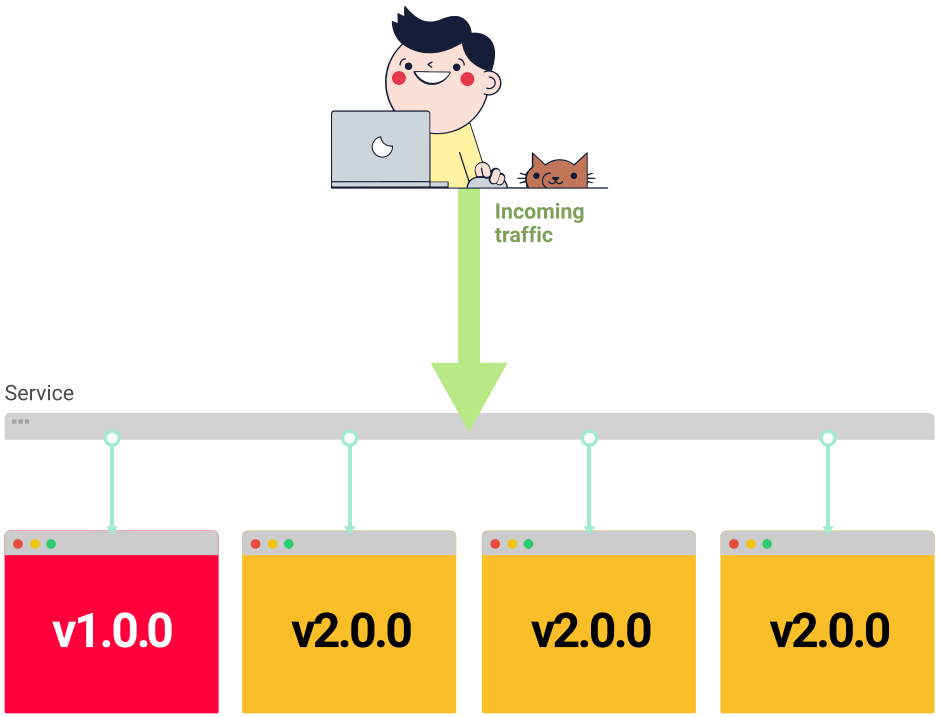


Fig. When you're confident you can rebalance a Canary deployment

In this example, the traffic is rebalanced in favour of the current version: 75% of the traffic hits the current version

and 25% the previous.

The process of routing the traffic progressively to newer Pods is manual, and you can fine-tune how long you wish to keep doing it.

In a Canary deployment, you want your Services to be able to route traffic to two sets of Pods: current and previous.

But how does a Service route traffic to the right Pod?

Using selectors and labels

Kubernetes Pods can be decorated with arbitrary key-value pairs called labels.

Labels are convenient because you could tag your Pods with a key-value pair such as `component: frontend` for a front-end component.

And you could use a similar strategy for a backend component: `component: backend`.

You can have any number of labels.

Indeed you could tag your Pods with a label for the version such as this:

pod.yaml

```
apiVersion: v1
```

```
kind: Pod
metadata:
  name: pod-with-labels
  labels:
    component: frontend
    version: "1.0.0"
spec:
  containers:
  - name: myapp-container
    image: busybox
```

Services can route traffic to Pods when they match a selector. A selector is used to target Pods with a specific label such as `component: frontend`.

service.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: my-service
spec:
  selector:
    component: frontend
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

The above Service routes traffic to all Pods that have a `component: frontend` label.
When a set of Pods share the same label, the Service routes traffic to all of them even if the Pods belong to different Deployments.

Using labels and selectors with Canary Deployments

You could use selectors and labels to create a Canary deployment and route traffic to two different sets of Pods.

You should create two Deployments: one for the existing app with Pods with a `version: 1.0.0` label and another for the new app with a `version: 2.0.0` label.

When you start, 100% of the traffic is routed to the existing application.

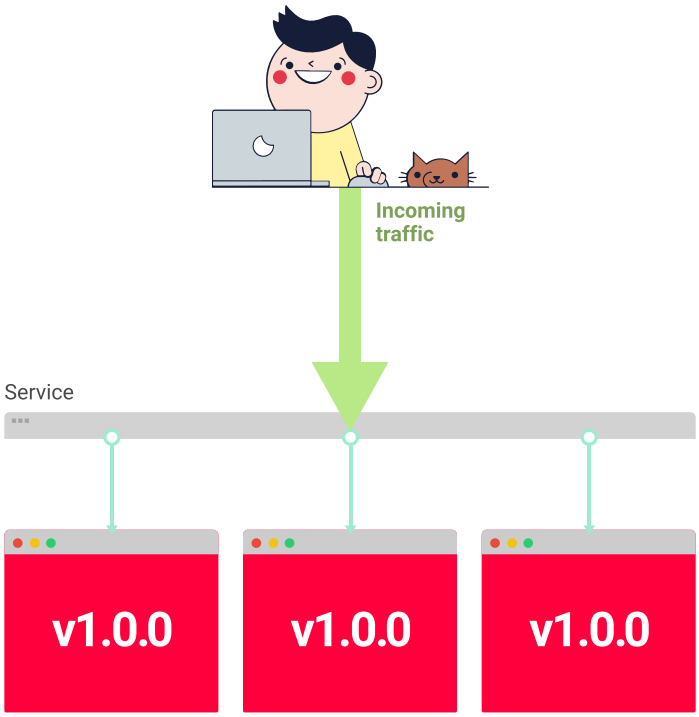


Fig. The Service route traffics to red Pods

The Service's selector points to `version: 1.0.0`.

service.yaml

```yaml
kind: Service
apiVersion: v1
metadata:
  name: canary-service
spec:
  selector:
    version: "1.0.0"
```

```
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

You now decide to route a fraction of the live traffic to the new version.

With a single instance of the current app and three instances of the previous one, 75% of the traffic will be routed to the existing app and only 25% to the latest version.

But you need a shared label for your Service's selector to route traffic to both.

You could add a label of `component: frontend` to both and have the Service target that name instead of the version.

service.yaml

```
kind: Service
apiVersion: v1
metadata:
  name: canary-service
spec:
  selector:
    component: frontend
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

As soon as you make the change, the Service routes the traffic to the current app (hopefully) one time every four visits.

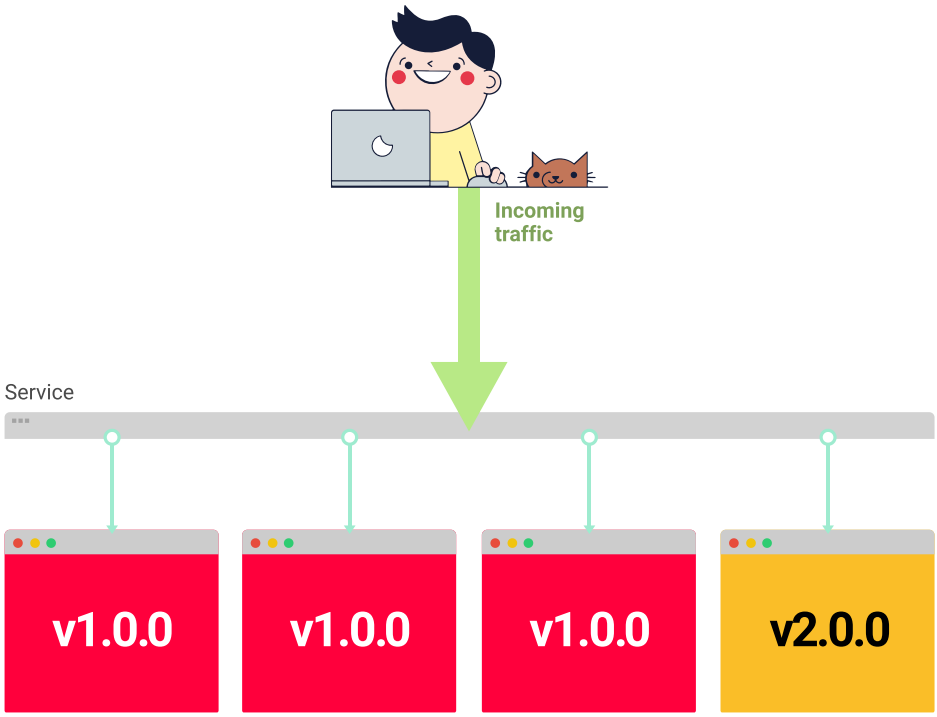


Fig. You have two Deployments, one Service and a shared label in a Canary deployment

If you're pleased with the result, you can gradually move your production traffic to the new version by increasing the replicas in the Deployment and decreasing the count in the previous.

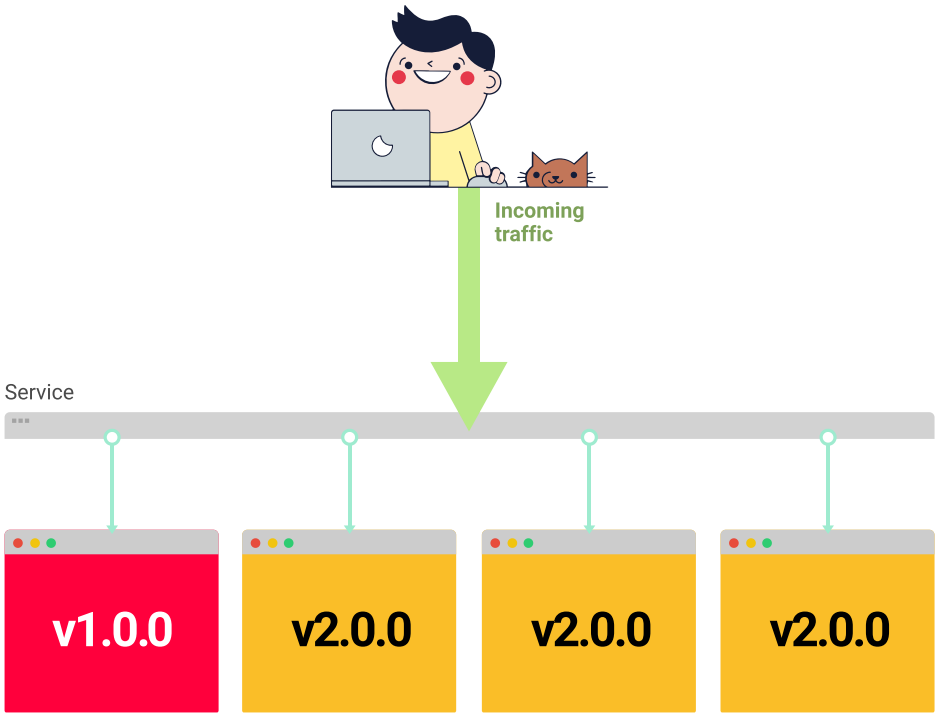


Fig. When you're confident you can rebalance a Canary deployment

When you're happy to switch all traffic, you can edit the Service and point to `version: 2.0.0` instead of the shared label.

service.yaml

```
kind: Service
```

```
apiVersion: v1
metadata:
  name: canary-service
spec:
  selector:
    version: "2.0.0"
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

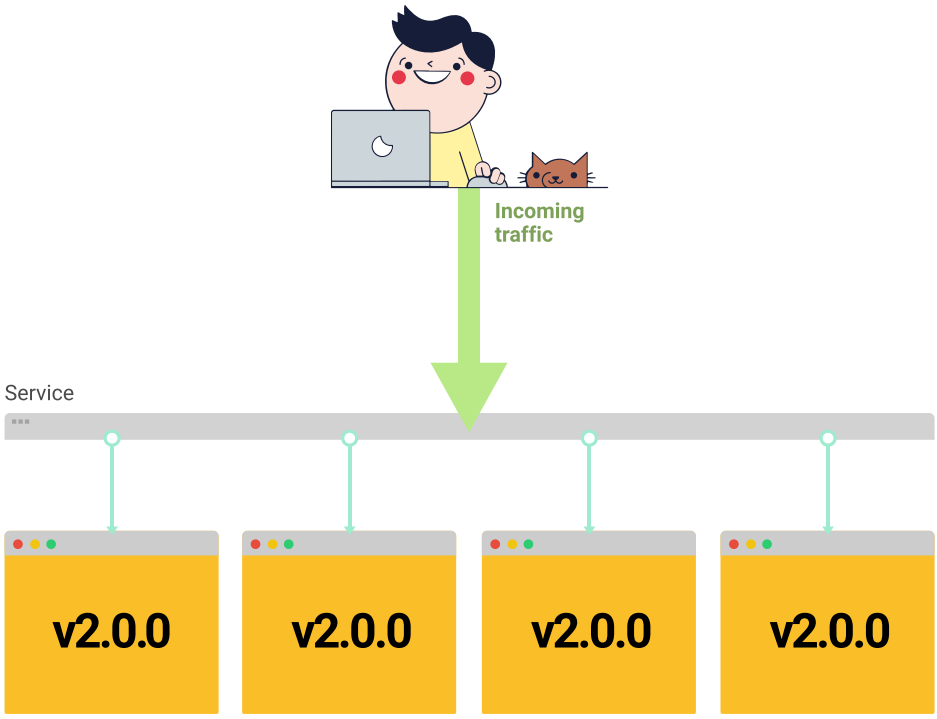


Fig. Moving all traffic to the current version

Services make for a great dynamic load balancer.

You can dynamically reconfigure the Service to route traffic to a set of Pods by tweaking the selector and the labels on your Pods.

Blue-green deployments

Rolling updates and Canary deployments are excellent strategies to introduce incremental updates.

However, there're times when you have to introduce breaking changes to your API or application.

In that case, you can't have two versions of your application live at the same time as it will break the existing contract with your users.

A more appropriate strategy to rollout breaking changes is to use a blue-green Deployment.

In a blue-green Deployment you create a copy of your application but with the newer version installed and then you make the existing Service point to the latest version.

You start with a Service and a Deployment.

The selector for the Pods is pointing to `version: 1.0.0`.

You then create a second Deployment with the newer app and where the Pods have a label of `version: 2.0.0`.

When you're ready, you change the selector in the Service from `version: 1.0.0` to `version: 2.0.0` and suddenly all of your traffic is served from the newer Deployment.

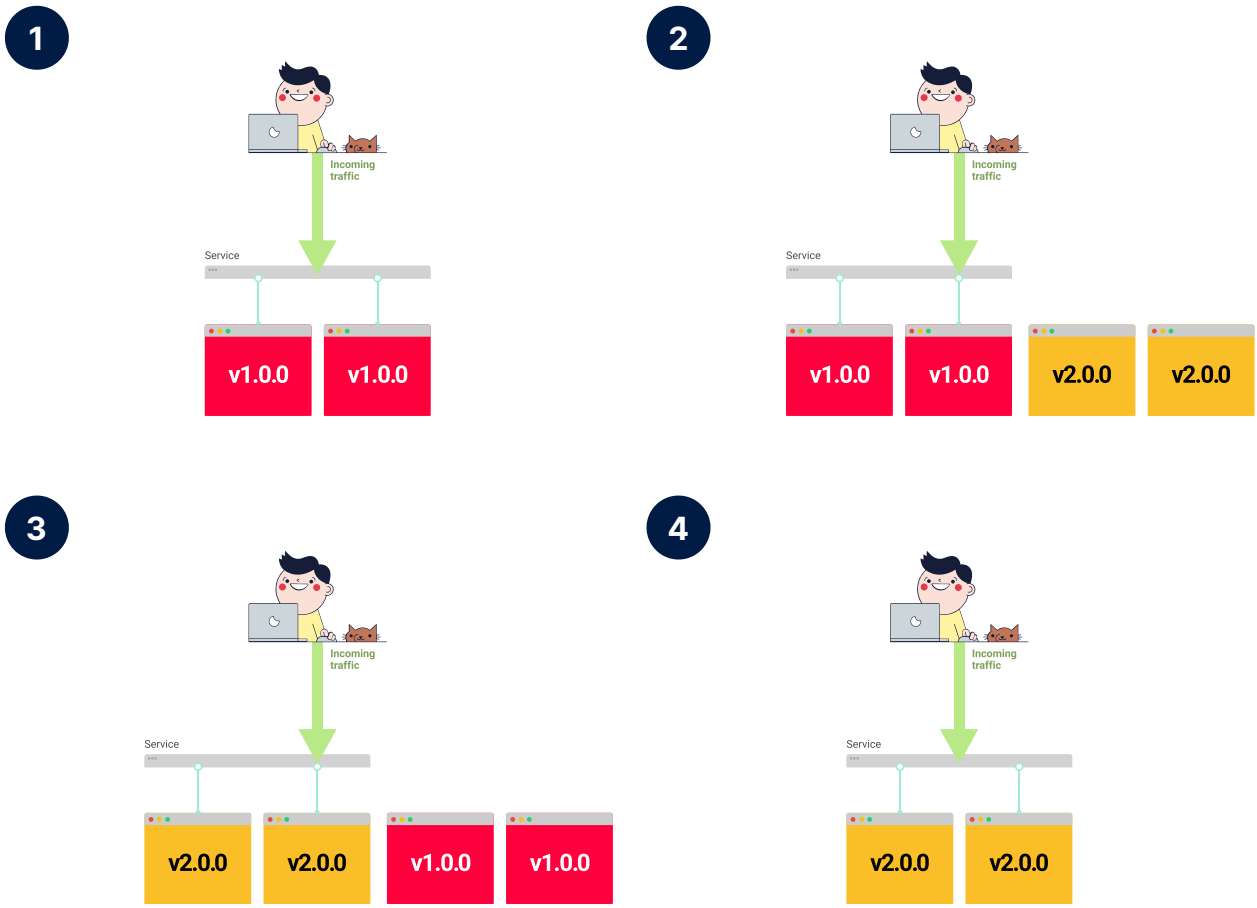


Fig. 1 You start with a single Deployment and Service. All traffic is routed to the previous version.

Fig. 2 You create another Deployment. An exact copy of the previous one.

Fig. 3 You change the selector in the Service to target the current Deployment.

Fig. 4 You dispose of the previous Deployment.

Don't forget to remove the Deployment for the previous app.

> When you execute a blue-green release with hundreds of Pods or more, you will have double the number of Pods until you destroy the previous Deployment. Make sure you have enough capacity.

Rolling back a Deployment

Even if you use techniques such as Rolling updates, Canary and Blue-green Deployments, there's still risk that your application doesn't work the way you expect.

When you introduce a change that breaks production, you should have a plan to roll back that change.

Kubernetes and `kubectl` offer a simple mechanism to roll

back changes to resources such as Deployments, StatefulSets and DaemonSets.

But before talking about rollbacks, you should learn an important detail about Deployments.

You learned how Deployments are responsible for gradually rolling out new versions of your Pods without causing any downtime.

You are also familiar with the fact that Kubernetes watches over the number of replicas in your deployment.

If you asked for 5 Pods but have only 4, Kubernetes creates one more.

If you asked for 4 Pods, but you have 5, Kubernetes deletes one of the running Pods.

Since the replicas is a field in the Deployment, you might be tempted to conclude that is the Deployment's job to count the number of Pods and create or delete them.

This is not the case, unfortunately.

Deployments delegate counting Pods to another component called the ReplicaSet.

Every time you create a Deployment, the deployment creates a ReplicaSet and delegates creating (and deleting) the Pods.

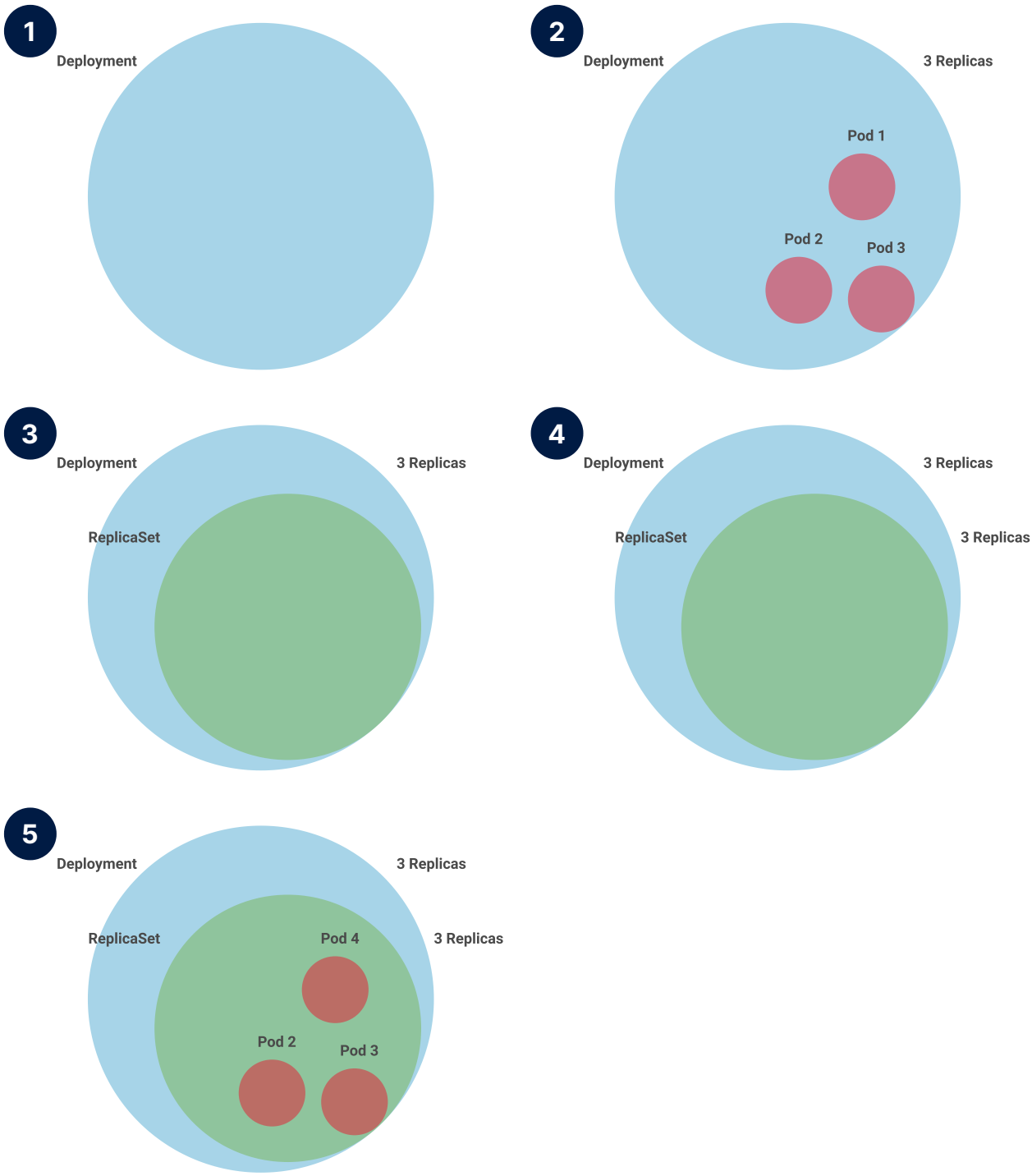


Fig. 1 Let's focus on a Deployment.

Fig. 2 You might be tempted to think that Deployments are in charge of creating Pods.

Fig. 3 The Deployment doesn't create Pods. Instead it creates another object called ReplicaSet.

Fig. 4 The Deployment passes the spec (which includes the replicas) to the ReplicaSet.

Fig. 5 The ReplicaSet is in charge of creating the Pods and watching over them.

But why isn't the Deployment creating the Pods?
Why does it have to delegate that task to someone else?

Let's consider the following scenario.

You have a Deployment with a container on version 1 and three replicas.

You change the spec for your template and upgrade your container from version 1 to version 2.

The ReplicaSet can hold only a single type of Pod.

So you can't have version 1 and version 2 of the Pods in the same ReplicaSet.

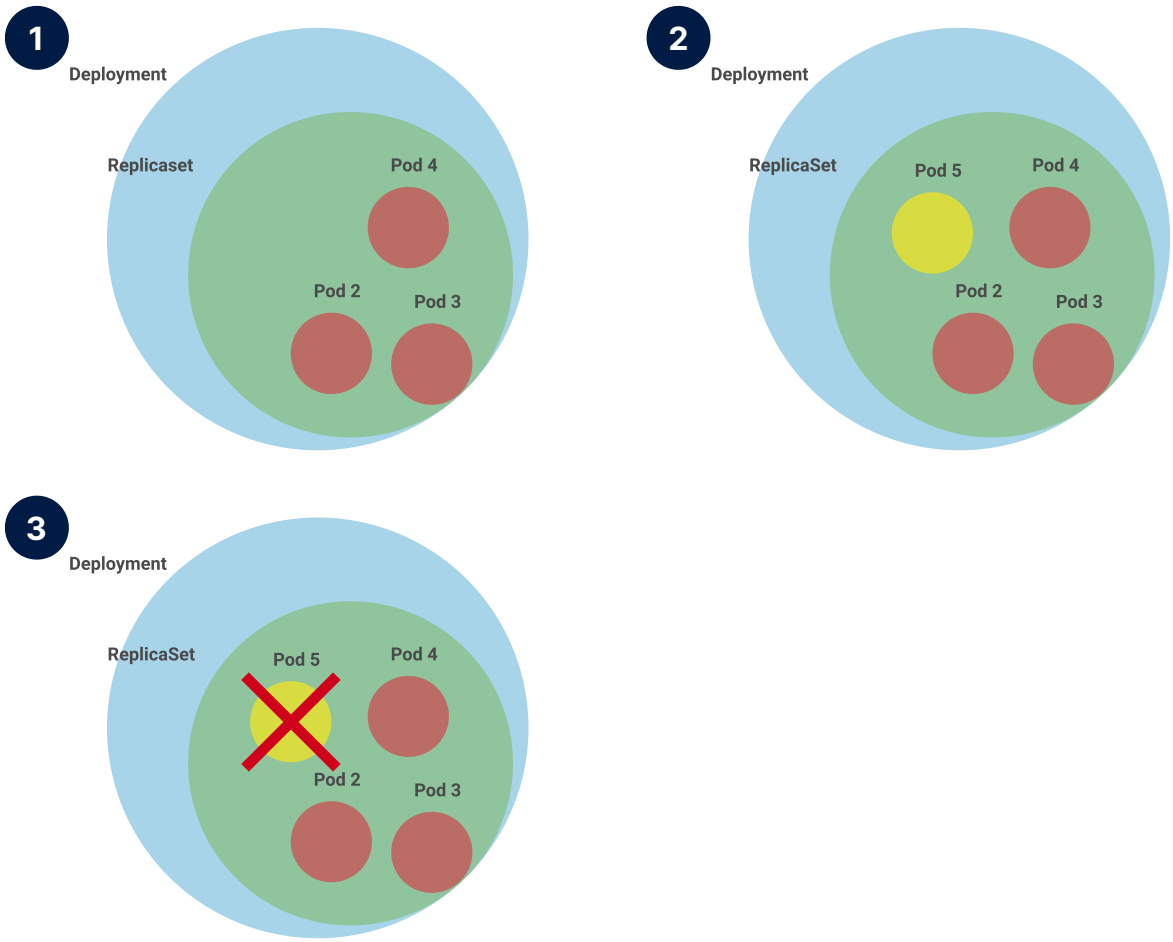


Fig. 1 Deployments create ReplicaSets that create Pods.

Fig. 2 Can you have two different Pods in the same ReplicaSet?

Fig. 3 ReplicaSets can only contain a single type of Pod. You can't use two different Docker images. *How can you deploy two versions of the app simultaneously?*

The Deployment knows that the two Pods can't coexist in the same ReplicaSet, so it creates a second ReplicaSet to hold

version 2.

Then gradually it decreases the count of replicas from the previous ReplicaSet and increases the count on the current one until the latter ReplicaSet has all the Pods.

> Please note that the behaviour for how many Pods are created and destroyed by the rolling update is controlled by the field `Deployment.spec.strategy.rollingUpdate`. You can learn about the settings with `kubectl explain Deployment.spec.strategy.rollingUpdate`.

In other words, the sole responsibility for the ReplicaSet is to count Pods.

Instead, the Deployment manages ReplicaSets and orchestrates the rolling update.

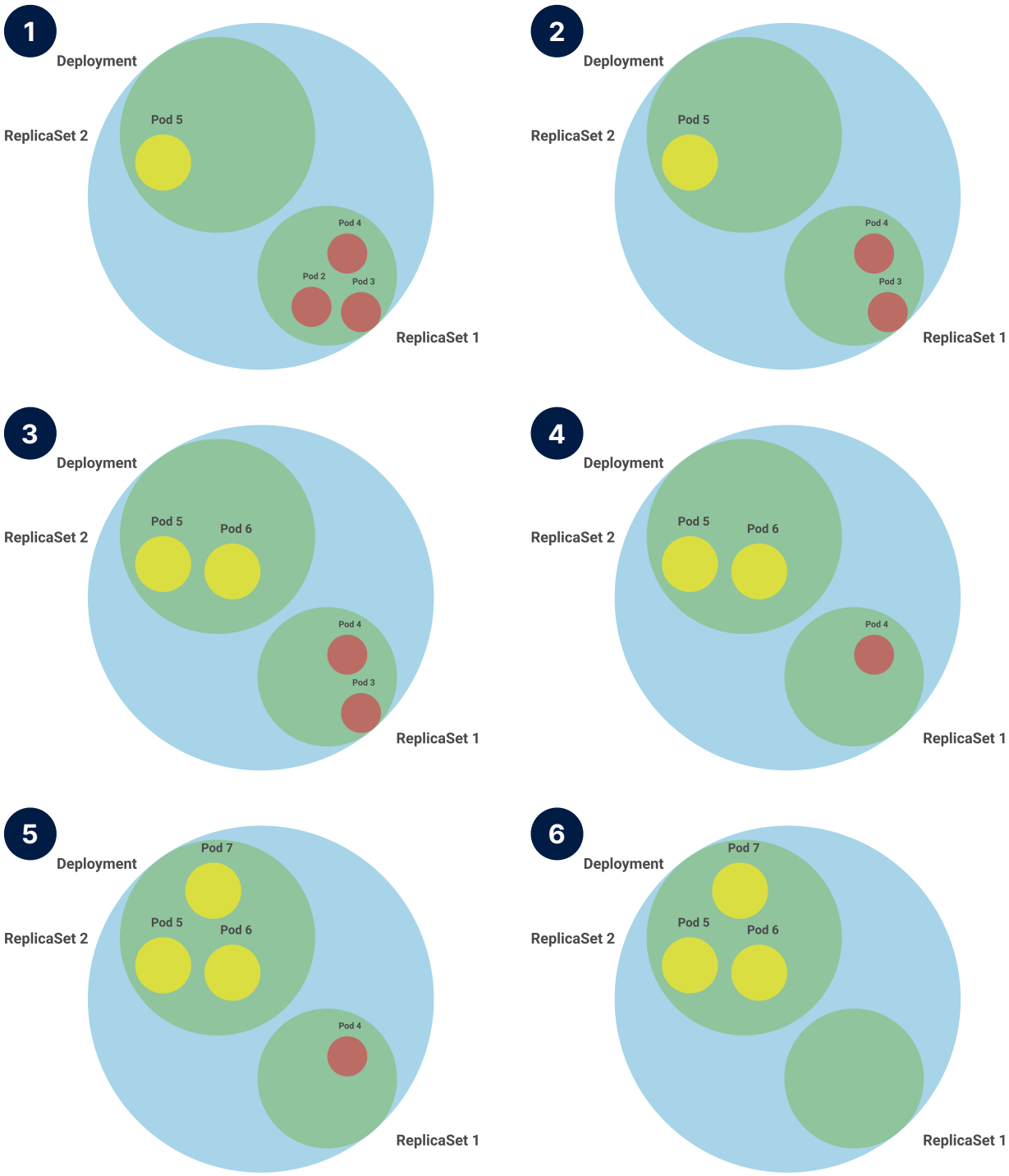


Fig. 1 The Deployment knows that you can't have different Pods in the same ReplicaSet. So it creates another ReplicaSet. It increases the number of replicas of the current ReplicaSet to one.

Fig. 2 And then it proceeds to decrease the replicas count in the previous ReplicaSet.

Fig. 3 The same process of increasing and decreasing Pods continues until all Pods are created on the current ReplicaSet.

Fig. 4 Please notice how you have two Pods templates and two ReplicaSets.

Fig. 5 Also, the traffic is hitting both the current and previous version of the app.

Fig. 6 After the rolling update is completed, **the previous ReplicaSet is not deleted.**

But what if you don't care about rolling updates and only wish for your Pods to be recreated when they are deleted?

Could you create a ReplicaSet without a Deployment?

Of course, you can.

Here's an example of a ReplicaSet.

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: example-replicaset
spec:
```

```
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: app
        image: learnk8s/hello:1.0.0
```

For reference, this is a Deployment that creates the ReplicaSet above:

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: app
        image: learnk8s/hello:1.0.0
```

Aren't those the same?

They are in this example.

However, in a Deployment, you can define properties such *how long to wait before considering the Pod live (*`minReadySeconds`*).*

The same property isn't available in the ReplicaSet.

> How do you know which properties are available? You can consult the official API.

In general, the YAML for the Deployment contains the ReplicaSet plus some additional details.

You can create the ReplicaSet with:

```bash
$ kubectl apply -f replicaset.yaml
```

There's something else worth noting about the ReplicaSets and Deployments.

When you upgrade your Pods from version 1 to version 2, the Deployment creates a new ReplicaSet and increases the count of replicas while the previous count goes to zero.

After the rolling update, the previous ReplicaSet is not deleted — not immediately at least.

Instead, it is kept around with a replicas count of 0.

If you try to execute another rolling update from version 2 to version 3, you might notice that at the end of the

upgrade, you have two ReplicaSets with a count of 0.

Why are the previous ReplicaSets not deleted or garbage collected?

Imagine that the current version of the container introduces a regression.

You probably don't want to serve unhealthy responses to your users, so you might want to roll back to a previous version of your app.

If you still have an old ReplicaSet, perhaps you could scale the current replicas to zero and increment the previous ReplicaSet count.

In other words, **keeping the previous ReplicaSets around is a convenient mechanism to roll back to a previously working version of your app.**

By default Kubernetes stores the last 10 ReplicaSets and lets you roll back to any of them.

But you can change how many ReplicaSets should be retained by changing the `spec.revisionHistoryLimit` in your Deployment.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-deployment
spec:
  replicas: 3
  revisionHistoryLimit: 100
  selector:
```

```
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: app
        image: learnk8s/hello:1.0.0
```

What about the previous ReplicaSets?
Could you list all the previous Replicasets that belong to a Deployment?
You can use the following command to inspect the history of your Deployment:

bash

```
$ kubectl rollout history deployment/app
```

And you can rollback to a specific version with:

bash

```
$ kubectl rollout undo deployment/app --to-revision=2
```

But how does the Deployment know their ReplicaSets?
Does it store the order in which ReplicaSets are created?
The ReplicaSets have random names with id such as `app-6ff88c4474`, so you should expect the Deployment to store

a reference to them.

Let's inspect the Deployment with:

```bash
$ kubectl get deployment app -o yaml
```

Nothing is looking like a list of previous 10 ReplicaSets.

Deployments don't hold a reference to their ReplicaSets.

At least not in the same YAML.

Instead, related ReplicaSets are retrieved comparing the template section in YAML.

Remember when you learnt that Deployments are ReplicaSets with some extra features?

Kubernetes uses that information to compare Deployments and ReplicaSets and make sure that they are related.

What about the order?

How do you know which one was the last ReplicaSet used? Or the third?

Kubernetes stores the revision in the `ReplicaSet.metatada.annotation`.

You can inspect the revision with:

```bash
$ kubectl get replicaset app-6ff88c4474 -o yaml
```

In the case below the revision is 3:

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: example-replicaset
  annotations:
    deployment.kubernetes.io/revision: "3"
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: app
        image: learnk8s/hello:1.0.0
```

So, what happens when you find a regression in the current release and decide to rollback to version 2 like so:

bash

```
$ kubectl rollout undo deployment/app --to-revision=2
```

- Kubectl finds the ReplicaSets that belong to the Deployment
- Each ReplicaSet has a revision number. Revision 2 is selected

- The current replicas count is decreased, and the count is gradually increased in the ReplicaSet belonging to revision 2
- The `deployment.kubernetes.io/revision` annotation is updated. The current ReplicaSet changes from revision 2 to 4

If before the undo you had three ReplicaSets with revision 1, 2 and 3, now you should have 1, 3 and 4.

There's a missing entry in the history: the revision 2 that was promoted to 4.

There's also something else that looks useful but doesn't work quite right.

The history command displays two columns: *Revision* and *Change-Cause*.

bash

```
$ kubectl rollout history deployment/app
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
```

While you're now familiar with the Revision column, you might be wondering what Change-Cause is used for — and why it's always set to `<none>`.

When you create a resource in Kubernetes, you can append the `--record` flag like so:

bash

```
$ kubectl apply -f deployment.yaml --record
```

When you do, Kubernetes adds an annotation to the resource with the command that generated it.

In the example above, the Deployment has the following metadata section:

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-deployment
  kubernetes.io/change-cause: kubectl apply --filename=deployment.yaml
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: app
        image: learnk8s/hello:1.0.0
```

Now, if you try to display the history again, you might notice that the same annotation is used in the rollout history command:

bash

```
$ kubectl rollout history app
```

```
REVISION  CHANGE-CAUSE
1         kubectl apply --filename=deployment.yaml --record=true
```

If you change the container image in the YAML file and apply the new configuration with:

bash

```
$ kubectl apply -f deployment.yaml --record
```

You should see the following new entry in the rollout history:

bash

```
$ kubectl rollout history deployment/app
REVISION  CHANGE-CAUSE
1         kubectl apply --filename=deployment.yaml --record=true
2         kubectl apply --filename=deployment.yaml --record=true
```

The `--record` command can be used with any resource type, but it has an effect only on Deployment, DaemonSet, and StatefulSet resources, i.e. resources that can be "rolled out" (see `kubectl rollout -h`).
But you should remember:

- The `--record` flag adds an annotation to the YAML resource, which can be changed at any time
- The rollout history command uses the value of this annotation to populate the Change-Cause table

- The annotation contains the last command only. If you create the resource and later use `kubectl scale --replicas=10 deploy/app --record` to scale it, only the scaling command is stored in the annotation.

Also, there is an ongoing discussion on deprecating the `--record` flag.

The feature provides little value for manual usage, but it still has some justification for automated processes as a simple form of auditing (keeping track of which commands caused which changes to a rollout).

You're ready to start practising with Rolling updates, Canary deployments, Blue-green deployments, Labels and Selectors and rollbacks!

Chapter 2

Before you start

For this module, you will need:

- Docker
- `minikube` and
- `kubectl`.

If this isn't the first time that you're running `minikube`, you can start fresh with:

INSTRUCTIONS FOR MACOS

bash

```
$ minikube delete && minikube start --driver=virtualbox
👍   Starting control plane node minikube in cluster minikube
🐳   Preparing Kubernetes v1.23.0 ...
🔎   Verifying Kubernetes components...
🌟   Enabled addons: default-storageclass, storage-provisioner
🏄   Done! kubectl is now configured to use "minikube" by default
```

INSTRUCTIONS FOR LINUX

bash

```
$ minikube delete && minikube start --drvier=virtualbox
👍   Starting control plane node minikube in cluster minikube
🐳   Preparing Kubernetes v1.23.0 ...
🔎   Verifying Kubernetes components...
🌟   Enabled addons: default-storageclass, storage-provisioner
🏄   Done! kubectl is now configured to use "minikube" by default
```

INSTRUCTIONS FOR WINDOWS

PowerShell — ☐ X

```
PS> minikube delete && minikube start --driver=hyperv
```

```
👍  Starting control plane node minikube in cluster minikube
🐋  Preparing Kubernetes v1.23.0 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

As soon as minikube starts, you can check that your cluster is ready with:

bash

```
$ kubectl cluster-info
Kubernetes master is running at https://192.168.64.20:8443
KubeDNS is running at <long url>
```

Familiarise with the apps

In this part of the course, you will practice different deployment strategies and observe how Kubernetes probes play a strategic role in rolling out a new version of your apps with zero downtime.

We packaged three applications for you:

- `learnk8s/deployment-strategies:v1`
- `learnk8s/deployment-strategies:v2`
- `learnk8s/deployment-strategies:v3`

Those are rather dull apps:

1. They display a static page with the version number.
2. They expose a `/health` and `/ready` endpoint.



Fig. Three apps

Unfortunately, those apps also somewhat problematic.

1. If you try to retrieve any endpoint within 5 seconds since the container started, the app crashes.
2. You can see the static page only 15 seconds after the container started. If you try to retrieve a URL before that, you will receive an error message.
3. The web server goes into a deadlock after 2 minutes. It will stop replying to your request.

And worse, you can't change the code for those apps.

So you will have to use all the tools at your disposal to fix it and keep it running.

It's worth inspecting the app and familiarising with the odd behaviour before deploying it into Kubernetes.

Let's start the first container locally with:

bash

```
$ docker run --rm -ti -p 8080:8080 learnk8s/deployment-strategies:v1
Listening on port: 8080
```

If you try to make a request to http://localhost:8080, you should notice that the container crashed.

bash

```
$ docker run --rm -ti -p 8080:8080 learnk8s/deployment-strategies:v1
Listening on port: 8080
Requested URL: /
called before startup: crashing
```

If you make a request after 5 seconds, but before 15, the error message is different:

bash

```
$ curl http://localhost:8080/ -v
< HTTP/1.1 500 Internal Server Error
< Content-Type: text/plain
< Date: Thu, 17 Jun 2021 04:50:21 GMT
< Connection: keep-alive
```

```
< Content-Length: 10
<
NOT READY
```

The application returns a 500 status code and the message `NOT READY`.
If you make a request after 15 seconds, you can finally see the static page:

bash

```
$ curl http://localhost:8080/
<html style="background-color:#FF003C">
  <head>
    <title>v1.0.0</title>
# truncated output
</html>
```

The same behaviour is present in `learnk8s/deployment-strategies:v2` and `learnk8s/deployment-strategies:v3`.
The only difference is that the static page includes a different colour and version.
You could think about those containers as three subsequent releases of the same application.

1. The first release has a plain red background.
2. The second release has a yellow background.
3. The third release has a green background.

Chapter 3

Rolling update strategy

Let's create a Deployment for the app.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
```

The above definition should look familiar:

- It's an object of kind `Deployment`.
- In the Pod template, a single container with the image `learnk8s/deployment-strategies:v1` is defined.
- The container exposes port 8080.
- There's a single replica.

You can submit the deployment to the cluster with:

bash

```
$ kubectl apply -f deployment.yaml
```

```
deployment.apps/app created
```

You can verify that a Pod was created with:

bash

```
$ kubectl get pods
NAME                   READY   STATUS    RESTARTS
app-56465cc99-wkv4n    1/1     Running   0
```

How could you visit the Pod, though?
You can expose the application with a Service.

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    name: app
```

Notice how:

- The Service's selector `name: app` matches the label on the Pod template.
- The `targetPort` matches the `containerPort` on the Deployment.

You can submit the service to the cluster with:

bash

```
$ kubectl apply -f service.yaml
service/entry-point created
```

And finally you can visit the app with:

bash

```
$ minikube service --url entry-point
http://192.168.64.18:32036
```

Depending on how quickly you went through the steps above, you might notice one of the following:

1. You can see a red static page.
2. The application replies with a `NOT READY` message.
3. The application can't be reached and you can't see anything.

If you notice the latter, it might be that the application enter in a dead-lock already.

You can inspect the logs for the single Pod to verify that it's the case:

bash

```
```

```
$ kubectl logs <pod id>
Server running at http://0.0.0.0:8080/
Started up
Requested URL: /
Ready
This will app will crash in 1 minute
Server crashed
```

You can "fix" the pod by deleting it.

bash

```
$ kubectl delete pod <pod id>
pod "app-56465cc99-wkv4n" deleted
```

If you issue the request again, you should see the web page.

But how could you automatically restart rather than issueing a `kubectl delete` *every time the web server crashes?*

Adding a Liveness probe

A liveness probe is designed to probe the container and check if the process is healthy.

If it isn't, it restarts the container.

You could use a liveness probe to automatically restart the container when the web server goes into a deadlock.

Let's amend the Deployment definition to include it:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

Let's break down the details:

- `httpGet` is the command that should be executed to inspect the endpoint. In this case is an HTTP probe, but you could have a TCP probe or execute a generic command in the container.
- `initialDelaySeconds` is the initial delay. The probe will start checking the container only after 6 seconds.
- `timeoutSeconds` specifies how long to wait before

- timing out the request.
- `periodSeconds` is the frequency used to check the probe. In this case, the `/health` endpoint is checked every 5 seconds.
- `successThreshold` is the number of successful attempts before you can consider the probe successful.
- `failureThreshold` is the number of failing attempts before the probe gives up.

You can submit the changes to the cluster with:

bash

```
$ kubectl apply -f deployment.yaml
deployment.apps/app configured
```

Let's test the new deployment:

bash

```
$ minikube service --url entry-point
http://192.168.64.18:32036
```

If you visit the URL, you will see any of the following:

1. The app can't be reached.
2. A message suggesting that the container is not ready.
3. The static red page.

You were probably expecting the static red page, but what about the other responses?

As soon as the container starts, traffic is immediately routed to it.

- If you make a request within 5 seconds since it started, the app will crash and it will be restarted.
- If you make a request after 5 seconds but before 15, the app is not ready.

Wouldn't be better if you could route the traffic to the app only after 15 seconds?

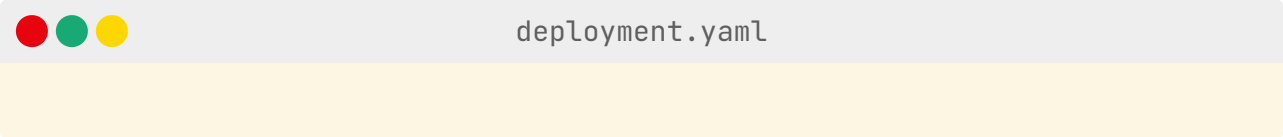
You can with the Readiness probe.

Adding a readiness probe

The readiness probe decides if the container is ready to receive traffic or not.

When it is ready, the probe attaches the container to the service.

So you could include a readiness probe in your Deployment like this:



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 3
```

As you might have noticed, most of the configuration for the readiness probe mimics the same properties of the liveness probe.

There's a notable change, though.

The `httpGet` probe checks the `/ready` and not the `/health` endpoint.

It makes sense:

- `/health` is the endpoint for the process. **The process could be healthy, but not ready to accept connections.**
- `/ready` is the endpoint for the traffic. **You might want to wait a bit longer after the process is healthy before you route traffic to it.**

Is the Readiness always executed after the Liveness probe?

Kubernetes doesn't care.

You can do a readiness check before the liveness or vice versa.

As far as Kubernetes is concerned, the two probes are independent.

However, as an engineer you might want to wait for the Liveness probe before you start checking the Readiness.

Let's submit the amended Deployment to the cluster:

bash

```
$ kubectl apply -f deployment.yaml
deployment.apps/app configured
```

And let's test it by visiting this URL:

bash

```
$ minikube service --url entry-point
```

```
http://192.168.64.18:32036
```

This time you should observe:

1. The app can't be reached or
2. The static red page.

You fixed the "NOT READY" messages, but what about the app not being available?

You are operating a single Pod.

If the Pod restarts and the readiness probe takes at least 15 seconds to attach the container to the Service, you have at least 15 seconds in which there is no Pod serving traffic.

Fortunately, you can easily fix this by increasing the number of replicas in your deployment.

Increase the replicas to 5:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 5
  selector:
    matchLabels:
      name: app
  template:
    metadata:
# truncated
```

Submit the deployment to the cluster with:

```bash
$ kubectl apply -f deployment.yaml
deployment.apps/app configured
```

Wait for the pods to start:

```bash
$ kubectl get pods
NAME                   READY   STATUS    RESTARTS
app-5b86746876-4lkk7   1/1     Running   1
app-5b86746876-7jcjw   1/1     Running   1
app-5b86746876-ddf24   1/1     Running   1
app-5b86746876-njwpq   1/1     Running   4
app-5b86746876-qtvhw   1/1     Running   1
```

From this point onwards, almost all requests return the static red page.

If you are very, very lucky, you might still notice timeouts.

But why?

The Readiness probe checks the container every 5 seconds (`periodSeconds`).

It will have to wait for 3 failed attempts (`failureThreshold`) before it decides to remove the container from the Service.

So it might take up to 15 seconds to detect a Pod with a malfunctioning web server.

Of course, you can fine-tune those values, but even if you select the very minimum for both settings (1 second and 1 attempt), there's always the chance that the application will crash in-between the interval.

You're minimising the chance of things going wrong and you have remedies, but things can still go wrong.

Let's reduce the failure threshold:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
```

```
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can submit the changes to the cluster with:

bash

```
$ kubectl apply -f deployment.yaml
deployment.apps/app configured
```

From this point onwards:

1. The application will be detached from the Service if the `/ready` endpoint doesn't return a successful message. This action could take up to 5 seconds.
2. The liveness probe will check the `/health` endpoint every 5 seconds. If the endpoint doesn't return a successful message, the container is restarted.

Congrats!

The application is far from great, but you managed to mitigate the risk of downtime.

Let's recap what you've done so far:

1. You created a Deployment and a Service for an application.

2. You set up a liveness probe to check if the containers needs restarting.
3. You configure a readiness probe to check when the container is ready to receive traffic.
4. You scale your deployment to 5 replicas.

In the next chapter you will upgrade your application to use a different container image.

Rolling updates

The applications deployed in the cluster might need to be updated from time to time.

By default, when you change the container image in the Deployment, Kubernetes executes a rolling update and replaces the previous Pods with others that run the newer container image.

With a rolling update strategy there is no downtime during the update process.

However the application must be architected to ensure that it can tolerate the pod being destroy and created at any time.

Thankfully, you can leverage all the effort you put in to tune Readiness and Liveness probe.

Monitoring the rolling updates

You will observe the Deployment as it rolls out to the newer version of the app.

But first, retrieve the URL for the `entry-point` service with:

INSTRUCTIONS FOR BASH

bash

```
$ SERVICE_URL=$(minikube service entry-point --url)
```

INSTRUCTIONS FOR POWERSHELL

PowerShell

```
PS> $Env:SERVICE_URL=(minikube service entry-point --url)
```

Next, you should monitor the response of the application.

Make a request to the app every second with:

INSTRUCTIONS FOR BASH

bash

```
$ while true; do printf %"s\n" $(curl -s $SERVICE_URL) | grep h1; sleep .5; done
```

INSTRUCTIONS FOR POWERSHELL

PowerShell

```
PS> while ($true) { (curl $Env:SERVICE_URL).Content | grep h1; sleep .5
```

You should be able to see the application replying with the version `1.0.0`.
Now open the deployment and change the container image to: `learnk8s/deployment-strategies:v2`.

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v2
        ports:
# truncated
```

Open a new terminal session and submit the deployment to the cluster with:

bash

```
$ kubectl apply -f deployment.yaml
deployment.apps/app configured
```

Did you notice anything?

In the previous terminal, Kubernetes starts updating the Pods.

Some of the traffic is served from version `2.0.0`.

You can inspect the status of the rollout with:

```bash
$ kubectl rollout status deploy/app
Waiting for deployment "app" rollout to finish: 1 out of 5 new replicas have been updated...
Waiting for deployment "app" rollout to finish: 2 out of 5 new replicas have been updated...
Waiting for deployment "app" rollout to finish: 3 out of 5 new replicas have been updated...
Waiting for deployment "app" rollout to finish: 4 out of 5 new replicas have been updated...
Waiting for deployment "app" rollout to finish: 2 old replicas are pending termination...
Waiting for deployment "app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "app" rollout to finish: 4 of 5 updated replicas are available...
deployment "app" successfully rolled out
```

Once the rollout is complete, the application should be running on version `2.0.0`.

You can verify it with:

```bash
$ minikube service entry-point --url
http://192.168.64.18:32036
```

No matter how many time you refrsh the browser, you always get version `2.0.0` as a response.

Notice how the rolling update strategy is part of the Deployment.

You didn't have to code any logic to execute a rolling update.

During a rolling update 25% of Pods are replaced at the

time.

If your deployment is 4 replicas, Kubernetes will execute a rolling update 1 Pod at the time.

If you have 20 replicas, Kubernetes will replace 5 Pods at the time.

You can configure how many Pods you want to be replaced with the following properties:

- `maxSurge`
- `maxUnavailable`

Now that you've mastered rolling updates and upgrade the app from version `1.0.0` to version `2.0.0`, *can you perform a rolling update and upgrade the application to version* `3.0.0` *?*

The image is for v3 is `learnk8s/deployment-strategies:v3`.

Tidying up

You should delete the existing Deployments with:

bash

```
$ kubectl delete deployment/app
deployment.apps "app" deleted
```

You can delete the Service with:

bash

```
$ kubectl delete service entry-point
service "entry-point" deleted
```

Chapter 4

Services and selectors

In Kubernetes, you can assign labels to resources such as Pods, Deployments or Services.

Labels are dynamic: you can add and remove them even after you created the resource.

In this section, you will learn how to use Service selectors to route traffic to a subset of Pods.

Deploying the applications

To start with, let's create two deployments: one for version `1.0.0` and one for version `2.0.0`.

Deploying version 1.0.0

You should create a `deployment-v1.yaml` file with the following content:

deployment-v1.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
spec:
  replicas: 4
  selector:
    matchLabels:
      name: app
  template:
```

```
    metadata:
      labels:
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-v1.yaml
deployment.apps/v1 configured
```

Deploying version 2.0.0

You should create a `deployment-v2.yaml` file with the

following content:

deployment-v2.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v2
spec:
  replicas: 4
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "2.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v2
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-v2.yaml
deployment.apps/v1 configured
```

You should verify that all pods are running with:

bash

```
$ kubectl get pods
NAME                  READY   STATUS    RESTARTS
v1-78c54955d5-bdkl4   0/1     Running   0
v1-78c54955d5-c4kgc   1/1     Running   0
v1-78c54955d5-kzzl5   0/1     Running   0
v1-78c54955d5-n8sgb   0/1     Running   0
v2-64f7cd6c6c-5jwlh   0/1     Running   1
v2-64f7cd6c6c-8kcxb   1/1     Running   0
v2-64f7cd6c6c-gfbcl   0/1     Running   1
v2-64f7cd6c6c-jq5qq   1/1     Running   1
```

You can also verify that all labels are applied correctly with:

bash

```
$ kubectl get pods --show-labels
NAME                  LABELS
v1-78c54955d5-bdkl4   name=app,version=1.0.0
v1-78c54955d5-c4kgc   name=app,version=1.0.0
v1-78c54955d5-kzzl5   name=app,version=1.0.0
v1-78c54955d5-n8sgb   name=app,version=1.0.0
v2-64f7cd6c6c-5jwlh   name=app,version=2.0.0
v2-64f7cd6c6c-8kcxb   name=app,version=2.0.0
v2-64f7cd6c6c-gfbcl   name=app,version=2.0.0
v2-64f7cd6c6c-jq5qq   name=app,version=2.0.0
```

Distributing traffic to one app at the time

If you wish to visit the Deployment *v1*, you have to create a Service.

Create a `service.yaml` file with the following content:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    version: "1.0.0"
```

Notice how the Service selector points to the label `version: "1.0.0"`.

You can create the resource with:

bash

```
$ kubectl apply -f service.yaml
service/entry-point created
```

You should visit the page and confirm that it works with:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

Until this point, everything is precisely the same as the last chapter.

You have a Deployment with 4 Pods, and the Service routes the traffic to those.

Now let's change the Service's selector to be `version: "2.0.0"`:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    version: "2.0.0"
```

And submit the resource with:

bash

```
$ kubectl apply -f service.yaml
service/entry-point configured
```

Retrieve the URL for the service with:

```bash
$ minikube service entry-point --url
http://192.168.64.18:32036
```

And visit it in your browser.

Kubernetes is serving version `2.0.0` of your application.

Excellent!

By changing the selector, you can target different Pods' labels and route the traffic elsewhere.

Notice how the Service is a convenient load balancer that can send traffic to Pods.

When the selector pointed to version `1.0.0`, the traffic was served from Pods from version `1.0.0`.

When the selector changed to point to version `2.0.0`, the traffic was served only from Pods with version `2.0.0`.

Distributing traffic to both apps

But both deployments share a label of type `name: app`.

If you change the selector to point to `name: app`, you can serve traffic from both deployments.

Let's amend the service:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    name: app
```

And submit the resource with:

bash

```
$ kubectl apply -f service.yaml
service/entry-point configured
```

If you refresh your browser multiple times, you should see two different pages for version `1.0.0` and `2.0.0` appearing intermittently.

It makes sense since this Service selector target both apps.

> If you're using Chrome and can't see the page intermittently appearing, please try again using `curl`. Chrome caches the response heavily.

There's another convenient command that you could use to save some time.
You can use labels to select pods with `kubectl get pods`:

bash

```
$ kubectl get pods -l version=1.0.0
NAME                  READY   STATUS
v1-78c54955d5-bdkl4   1/1     Running
v1-78c54955d5-c4kgc   1/1     Running
v1-78c54955d5-kzzl5   1/1     Running
v1-78c54955d5-n8sgb   1/1     Running
$ kubectl get pods -l version=2.0.0,name=app
NAME                  READY   STATUS
v2-64f7cd6c6c-5jwlh   0/1     Running
v2-64f7cd6c6c-8kcxb   0/1     Running
v2-64f7cd6c6c-gfbcl   0/1     Running
v2-64f7cd6c6c-jq5qq   0/1     Running
```

The `-l` flag is helpful to test what Pod could receive traffic for a given selector.

Labels challenge

Let's create a third deployment:

deployment-v3.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: v3
spec:
  replicas: 4
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "3.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v3
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-v3.yaml
deployment.apps/v3 configured
```

Can you change the Service selector so that the traffic is routed only to version `2.0.0` *and* `3.0.0` *?*

Tidying up

You should delete the existing Deployments with:

bash

```
$ kubectl delete deployment/v1
deployment.apps "v1" deleted
$ kubectl delete deployment/v2
deployment.apps "v2" deleted
$ kubectl delete deployment/v3
deployment.apps "v3" deleted
```

You can delete the Service with:

bash

```
$ kubectl delete service entry-point
service "entry-point" deleted
```

Chapter 5

Canary deployments

Canary deployments are helpful if you wish to test a feature with a small subset of your users.

The strategy consists of creating a second Deployment and routing a smaller part of the production traffic to it.

Make sure you don't have any Deployment or Service deployed in your cluster before you test the canary deployment.

You can list the Deployments in your cluster with:

bash

```
$ kubectl get deployments
No resources found in default namespace.
```

And you can delete a single deployment with:

bash

```
$ kubectl delete deployment <deployment_name>
```

You can list the Services in the cluster with:

bash

```
$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP
```

> Please don't delete the `kubernetes` Service.

You can delete a single service with:

```bash
$ kubectl delete service <service_name>
```

Let's start.

Deploying the production application

You should deploy version `1.0.0` of your application. Create a `deployment-prod.yaml` file with the following content:

deployment-prod.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
```

```
  template:
    metadata:
      labels:
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource in Kubernetes with:

bash

```
$ kubectl apply -f deployment-prod.yaml
deployment.apps/v1 configured
```

You can verify that three Pods are running with:

bash

```
$ kubectl get pods
NAME                  READY   STATUS
v1-78c54955d5-9mfvd   1/1     Running
v1-78c54955d5-gfdjf   1/1     Running
v1-78c54955d5-p55v9   1/1     Running
```

Create a `service.yaml` file with the following content:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    version: "1.0.0"
```

You can create the resource with:

bash

```
$ kubectl apply -f service.yaml
service/entry-point created
```

You can verify that your service was exposed correctly with:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

If you visit that URL, you should see the app with a red background.

You can think about this deployment as your production version.

It has three Pods, and it serves live traffic.

Deploying the canary

If you want to test how a newer version of the app copes with live traffic, you could create another Deployment.

Create a `deployment-canary.yaml` file with the following content:

deployment-canary.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v2
spec:
  replicas: 1
  selector:
    matchLabels:
      name: app
  template:
```

```
    metadata:
      labels:
        name: app
        version: "2.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v2
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

Notice how the second deployment uses the `learnk8s/deployment-strategies:v2` and has a single replica.

You can create the resource with:

bash

```
$ kubectl apply -f deployment-canary.yaml
deployment.apps/v2 configured
```

You should verify that four Pods are running:

- Three Pods run version `1.0.0`.
- One Pod runs version `2.0.0`.

All four pods share the same `name: app` label.
Let;s verify that with:

bash

```
$ kubectl get pods --show-labels
NAME                  READY   STATUS    LABELS
v1-78c54955d5-9mfvd   1/1     Running   name=app,version=1.0.0
v1-78c54955d5-gfdjf   1/1     Running   name=app,version=1.0.0
v1-78c54955d5-p55v9   1/1     Running   name=app,version=1.0.0
v2-64f7cd6c6c-6wrc4   1/1     Running   name=app,version=2.0.0
```

Please notice that the Service's selector points to `version: "1.0.0"` so only the production app receives traffic.
The single canary Pod is *Running*, but none of the traffic reaches it.

Monitoring the live traffic

Before you start sending traffic to the canary Pod, let's start monitoring the deployment and simulating live traffic.
Retrieve the URL for the current service with:

INSTRUCTIONS FOR BASH

```bash
$ SERVICE_URL=$(minikube service entry-point --url)
```

INSTRUCTIONS FOR POWERSHELL

```PowerShell
PS> $Env:SERVICE_URL=(minikube service entry-point --url)
```

Next, you should monitor the response of the application. Make a request to the app every second with:

INSTRUCTIONS FOR BASH

```bash
$ while true; do printf %"s\n" $(curl -s $SERVICE_URL) | grep h1; sleep .5; done
```

INSTRUCTIONS FOR POWERSHELL

```PowerShell
PS> while ($true) { (curl $Env:SERVICE_URL).Content | grep h1; sleep .5
```

You should be able to see the application replying with `1.0.0` all the times.
Only version `1.0.0` of the application is receiving traffic.

Exposing the canary deployment

Let's route some traffic to the canary deployment.

In another terminal change the Service to point to route traffic with both apps:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    name: app
```

Since all the Pods share the same label (`name: app`), the traffic is distributed across the four Pods.

And since there're three replicas of the application at version `1.0.0` and one for the canary, you should see version `1.0.0` 3 out of 4 times when you refresh the page.

You can verify that by exposing the application with:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

If you open that URL in your browser and keep refreshing, you should observe a 75%-25% split between version `1.0.0` and `2.0.0`.

Balancing the split

Let's assume that there are no issues with the canary deployment.

The next step is to switch the live traffic in full to the new version.

Let's start with scaling the canary deployment to 3 replicas with:

deployment-canary.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v2
spec:
  replicas: 3
  selector:
    matchLabels:
# truncated output
```

You can apply the changes with:

bash

```
$ kubectl apply -f deployment-canary.yaml
deployment.apps/v2 configured
```

If you go back to your browser and keep refreshing multiple times, you should see version `1.0.0` and `2.0.0` appearing with the same frequency (50%-50% split).

Excellent!

The last step in the process is deleting the old deployment.

bash

```
$ kubectl delete deployment/v1
deployment.apps "v1" deleted
```

Now all the traffic reaches version `2.0.0`.

Congrats, you just completed a canary deployment.

1. You tested the initial deployment with a subset of the production traffic and
2. Slowly increased the number of replicas and replaced the old deployment.

Multiple canaries challenge

Let's recreate the first deployment.

deployment-prod.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource in Kubernetes with:

bash

```
$ kubectl apply -f deployment-prod.yaml
deployment.apps/v1 configured
```

And create a third deployment too:

deployment-v3.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v3
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "3.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v3
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
```

```
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the third deployment with:

bash

```
$ kubectl apply -f deployment-v3.yaml
deployment.apps/v3 configured
```

You should have 3 deployments with 9 Pods running:

bash

```
$ kubectl get deployments
NAME   READY   UP-TO-DATE   AVAILABLE
v1     3/3     3            3
v2     3/3     3            3
v3     3/3     3            3
$ kubectl get pods --show-labels
NAME                  READY   STATUS    LABELS
v1-78c54955d5-9mfvd   0/1     Running   name=app,version=1.0.0
v1-78c54955d5-gfdjf   1/1     Running   name=app,version=1.0.0
v1-78c54955d5-p55v9   0/1     Running   name=app,version=1.0.0
v2-64f7cd6c6c-5wf2s   1/1     Running   name=app,version=2.0.0
v2-64f7cd6c6c-6wrc4   1/1     Running   name=app,version=2.0.0
v2-64f7cd6c6c-h4rsb   1/1     Running   name=app,version=2.0.0
v3-8c74d5cc-5pd6j     1/1     Running   name=app,version=3.0.0
v3-8c74d5cc-6mt5q     1/1     Running   name=app,version=3.0.0
v3-8c74d5cc-ngcpv     1/1     Running   name=app,version=3.0.0
```

Can you change the YAML definitions for Deployments and Service so that the traffic reaches version `1.0.0`, `2.0.0` *and* `3.0.0` *with the following split?*

- `1.0.0` 50%
- `2.0.0` 25%
- `3.0.0` 25%

Tidying up

You should delete the existing Deployments with:

bash

```
$ kubectl delete deployment/v1
$ deployment.apps "v1" deleted
kubectl delete deployment/v2
deployment.apps "v2" deleted
kubectl delete deployment/v3
deployment.apps "v3" deleted
```

You can delete the Service with:

bash

```
$ kubectl delete service entry-point
service "entry-point" deleted
```

Chapter 6

Blue-green deployments

Blue-green deployment is a technique that reduces downtime and risk by running two identical production environments called Blue and Green.

At any time, only one of the environments receives traffic.

Before you continue, make sure you don't have any Deployment or Service deployed in your cluster.

You can list the Deployments in your cluster with:

bash

```
$ kubectl get deployments
No resources found in default namespace.
```

And you can delete a single deployment with:

bash

```
$ kubectl delete deployment <deployment_name>
```

You can list the Services in the cluster with:

bash

```
$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP
```

> Please don't delete the `kubernetes` Service.

You can delete a single service with:

bash

```
$ kubectl delete service <service_name>
```

Let's start.

Deploying the production application

The main web application receiving production traffic runs on version `1.0.0`.

Create a `deployment-blue.yaml` file with the following content:

deployment-blue.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
```

```
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-blue.yaml
deployment.apps/v1 configured
```

You can verify that three Pods are running with:

bash

```
```

```
$ kubectl get pods
NAME                  READY   STATUS    RESTARTS
v1-78c54955d5-bdkl4   1/1     Running   0
v1-78c54955d5-c4kgc   1/1     Running   0
v1-78c54955d5-kzzl5   1/1     Running   0
```

And create a `service-blue-green.yaml` file with the following content:

service-blue-green.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    version: "1.0.0"
```

You can create the resource with:

bash

```
$ kubectl apply -f service-blue-green.yaml
service/entry-point created
```

You can verify that your service is exposed correctly with:

bash

```
```

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

If you visit the URL in your browser, you will see the app on version `1.0.0`.

Monitoring

Before you switch the traffic to version `2.0.0`, let's start monitoring the deployment and simulating live traffic. Retrieve the URL for the current service with:

INSTRUCTIONS FOR BASH

bash

```
$ SERVICE_URL=$(minikube service entry-point --url)
```

INSTRUCTIONS FOR POWERSHELL

PowerShell

```
PS> $Env:SERVICE_URL=(minikube service entry-point --url)
```

Next, you should monitor the response of the application. Make a request to the app every second with:

INSTRUCTIONS FOR BASH

```bash
$ while true; do printf %"s\n" $(curl -s $SERVICE_URL) | grep h1; sleep .5; done
```

INSTRUCTIONS FOR POWERSHELL

```
PS> while ($true) { (curl $Env:SERVICE_URL).Content | grep h1; sleep .5
```

You should be able to see the application replying with `1.0.0` all the times.

Excellent!

It's time to prepare for the switch.

Create another deployment for a newer version of the application: version `2.0.0`.

You should create a `deployment-green.yaml` file with the following content:

deployment-green.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v2
spec:
  replicas: 1
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "2.0.0"
```

```
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v2
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

Submit the resource to the cluster with:

bash

```
$ kubectl apply -f deployment-green.yaml
deployment.apps/v2 configured
```

To route all the traffic from version `1.0.0` to `2.0.0` and achieve a blue-green deployment, open another terminal.

Update your service to point to version `2.0.0`:

service-blue-green.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    version: "2.0.0"
```

Apply the change with:

bash

```
$ kubectl apply -f service-blue-green.yaml
service/entry-point created
```

Verify that the application receiving the traffic runs on version `2.0.0`:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

In the other terminal session, you should notice a sharp change in responses from version `1.0.0` to `2.0.0`.

You just executed a blue-green deployment.

You created an exact copy of production with some new changes (version `2.0.0`) and reconfigured the load balancer

(Service) to point to the new stack.

Rainbow deployment challenge

Let's create a third deployment:

deployment-v3.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v3
spec:
  replicas: 4
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "3.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v3
        ports:
          - containerPort: 8080
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 6
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
        readinessProbe:
          httpGet:
            path: /ready
            port: 8080
          initialDelaySeconds: 15
```

```
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 1
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-v3.yaml
deployment.apps/v3 configured
```

Can you upgrade the application to version `3.0.0` *using a blue-green deployment?*

Tidying up

You should delete the existing Deployments with:

bash

```
$ kubectl delete deployment/v1
deployment.apps "v1" deleted
$ kubectl delete deployment/v2
deployment.apps "v2" deleted
$ kubectl delete deployment/v3
deployment.apps "v3" deleted
```

You can delete the Service with:

bash

```
$ kubectl delete service entry-point
service "entry-point" deleted
```

Chapter 7

Rolling back a deployment

This chapter will explore what you can do when you notice that a Deployment isn't functioning correctly.

Before you continue, make sure you don't have any Deployment or Service deployed in your cluster.

You can list the Deployments in your cluster with:

bash

```
$ kubectl get deployments
No resources found in default namespace.
```

And you can delete a single deployment with:

bash

```
$ kubectl delete deployment <deployment_name>
```

You can list the Services in the cluster with:

bash

```
$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP
```

> Please don't delete the `kubernetes` Service.

You can delete a single service with:

```bash
$ kubectl delete service <service_name>
```

Let's start.

Creating a deployment

You should deploy version `1.0.0` of your application. Create a `deployment-prod.yaml` file with the following content:

deployment-prod.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v1
        ports:
```

```
        - containerPort: 8080
      livenessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 6
        timeoutSeconds: 1
        periodSeconds: 5
        successThreshold: 1
        failureThreshold: 1
      readinessProbe:
        httpGet:
          path: /ready
          port: 8080
        initialDelaySeconds: 15
        timeoutSeconds: 1
        periodSeconds: 5
        successThreshold: 1
        failureThreshold: 1
```

You can create the resource with:

bash

```
$ kubectl apply -f deployment-prod.yaml
deployment.apps/v1 configured
```

You can verify that three Pods are running with:

bash

```
$ kubectl get pods
NAME                  READY   STATUS    RESTARTS
v1-78c54955d5-bdkl4   1/1     Running   0
v1-78c54955d5-c4kgc   1/1     Running   0
v1-78c54955d5-kzzl5   1/1     Running   0
```

And create a `service.yaml` file with the following content:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: entry-point
spec:
  ports:
  - port: 80
    targetPort: 8080
  type: NodePort
  selector:
    name: app
```

You can create the resource with:

bash

```
$ kubectl apply -f service.yaml
service/entry-point created
```

You can verify that your service is exposed correctly with:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

If you visit the URL in your browser, you will see the app on version `1.0.0`.

Now let's execute a rolling update and upgrade the container image to: `learnk8s/deployment-strategies:v2`.

deployment-prod.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
spec:
  replicas: 3
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v2
        ports:
# truncated
```

Apply the changes with:

bash

```
$ kubectl apply -f deployment-prod.yaml
deployment.apps/v1 configured
```

You can verify that rollout was executed correctly with:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

If you visit the URL in your browser, you will see the app on version `2.0.0`.

What happens if rolling out to version `2.0.0` *is a mistake?*

Kubernetes a few convenient commands to inspect the rollout.

You can inspect the rollout history with:

bash

```
$ kubectl rollout history deployment v1
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

You deployed twice, so the table has two rows.

But what is the `CHANGE-CAUSE` *column?*

You can decorate each deployment with an annotation and Kubernetes will automatically insert that value in the table.

Let's try that by rolling out version `3.0.0`.

Amend the deployment with:

deployment-prod.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1
  annotations:
    kubernetes.io/change-cause: "Upgraded to version 3"
spec:
  replicas: 3
  selector:
```

```
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
        version: "1.0.0"
    spec:
      containers:
      - name: web
        image: learnk8s/deployment-strategies:v3
        ports:
# truncated
```

And apply the changes with:

bash

```
$ kubectl apply -f deployment-prod.yaml
deployment.apps/v1 configured
```

You can verify that the deployment is successful by refreshing your browser.

If you inspect the history for the deployment, you should see the following:

bash

```
$ kubectl rollout history deployment v1
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         Upgraded to version 3
```

Excellent!

What if you want to roll back to a previous deployment?

Let's rollback the Deployment to version `2.0.0`:

bash

```
$ kubectl rollout undo deployment v1
deployment.apps/v1 rolled back
```

You should verify that the application receiving traffic runs on version `2.0.0`:

bash

```
$ minikube service entry-point --url
http://192.168.64.18:32036
```

Visit the URL in the browser, and you should observe version `2.0.0` appearing.

Great!

You deployed an upgrade (version `3.0.0`), realised it wasn't what you wanted and rolled back to the original state (version `2.0.0`).

Let's inspect the rollout history again with:

bash

```
$ kubectl rollout history deployment v1
REVISION  CHANGE-CAUSE
1         <none>
3         Upgraded to version 3
4         <none>
```

Notice how the revision number increments even when you rolled back.

But what is the Revision column about?

Every time you create a Deployment, the deployment makes a ReplicaSet and delegates creating (and deleting) the Pods to it.

The Deployment doesn't create the Pods; the ReplicaSet does.

Every time you execute a rolling update:

1. The Deployment creates a new ReplicaSet.
2. Decreases the replica count on the previous ReplicaSet.
3. Increases the replica count on the current ReplicaSet.

The last two steps are repeated until the previous ReplicaSet is scaled to zero.

Since you have executed three rolling updates, you should have at least 3 ReplicaSets in your cluster.

Let's verify that with:

bash

```
$ kubectl get replicasets
NAME            DESIRED   CURRENT   READY
v1-57cbf4bb76   3         3         3
v1-78c54955d5   0         0         0
v1-859959458c   0         0         0
```

There are three ReplicaSets, but how come only one has

Pods?

That's the current ReplicaSets.

All the other ReplicaSets represents earlier deployments scaled to zero.

If you inspect them, you can notice that they contain previous container images:

bash

```
$ kubectl get replicaset v1-78c54955d5 -o yaml | grep image:
      - image: learnk8s/deployment-strategies:v1
$ kubectl get replicaset v1-859959458c -o yaml | grep image:
      - image: learnk8s/deployment-strategies:v3
```

The `REVISION` column is populated by looking at the `revision` property on the ReplicaSet.

bash

```
$ kubectl get replicaset v1-78c54955d5 -o yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  annotations:
    deployment.kubernetes.io/desired-replicas: "3"
    deployment.kubernetes.io/max-replicas: "4"
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: "2021-06-24T08:17:24Z"
  generation: 4
# truncated output
```

Excellent!

Now looking at the rollout history table should make more

sense:

```bash
$ kubectl rollout history deployment v1
REVISION  CHANGE-CAUSE
1         <none>
3         Upgraded to version 3
4         <none>
```

Lastly, you can revert to a specific point in time with `--to-revision`.
You can revert to the first deployment with:

```bash
$ kubectl rollout undo deployment v1 --to-revision=1
deployment.apps/v1 rolled back
```

Well done for reaching the end of Deployment strategies.
You're now an expert on zero-downtime deployments 🎉